

Danmarks
Tekniske
Universitet



Master Thesis

Added Resistance of a Ship Navigating in Ice Floes

AUTHOR

Maxime BECUWE – s243496 - s243496@dtu.dk

SUPERVISORS

DTU

Jens Honoré Walther – Civil and Mechanical Engineering
Yanlin Shao – Civil and Mechanical Engineering

OSK Design

Miguel Brito – Naval Architect
Nestor Gómez – Naval Architect M.Sc

26 June 2026

Abstract

The prediction of ship resistance in ice-covered waters is a critical challenge for the maritime industry, as Arctic shipping routes become increasingly accessible due to climate change. Model-scale towing tank tests, while valuable, are expensive and of limited repeatability when ice floes are involved. Numerical simulation offers a cheaper and more flexible alternative.

The aim of this thesis is to use the commercial CFD–DEM software Simcenter STAR-CCM+ to calculate the added resistance of a ship advancing through a field of floating ice floes. The fluid flow is governed by the RANS equations and the free surface is captured using the Volume of Fluid (VOF) method. Ice floes are modelled as discrete cylindrical DEM particles, enabling explicit resolution of hull–floe and floe–floe collisions. A large part of the work has been focused on the generation of realistic ice-floe fields, the calibration of contact parameters, and the correction of hydrodynamic forces acting on the floes.

The CFD–DEM setup is validated against published experimental data using the KCS (KRISO Container Ship), over a range of speeds ($F_r = 0.06$ – 0.18) and ice concentrations of 60%, 70%, and 80%.

The results show that the model predicts the added ice resistance with relative errors below 10% at high speeds and moderate concentrations. At low speeds and high concentrations, the model underestimates the resistance due to premature floe rotation. Despite these limitations, the model demonstrates that CFD–DEM can be used as a reliable tool to investigate the added resistance of ships navigating in ice floes.

Contents

Nomenclature	1
1 Introduction	2
1.1 Research motivation	2
1.2 Studies of ship–ice interaction	4
1.3 Sea-ice characterisation and ice-ice interaction	4
1.4 Objectives and scope	5
2 Numerical Methods	6
2.1 Fluid Solver	6
2.1.1 Navier–Stokes Equations	6
2.1.2 Turbulence Modelling	6
2.1.3 Free-Surface Modelling	7
2.2 Ice Floe Modelling	8
2.2.1 CFD–DEM Coupling	8
2.2.2 Lagrangian Description and DEM Formulation	8
2.2.3 Governing Equations for the DEM Particles	8
2.2.3.1 Contact force model.	9
2.2.4 DEM solver parameters	11
2.2.5 Limitations of the CFD–DEM Model	11
2.2.5.1 Hydrodynamic Torque and Added Moment of Inertia.	12
2.2.5.2 Particle Drag Torque.	13
2.3 Summary of Numerical Parameters	15
2.3.1 Fluid Properties	15
2.3.2 Ice Floe Geometry and Properties	15
2.3.3 DEM Hydrodynamic Coefficients	15
2.3.4 DEM Contact Model Parameters	16
2.3.5 Numerical Setup	16
3 Computational Setup	17
3.1 KCS Geometry	17
3.2 Domain and Boundary Conditions	17
3.3 Mesh	19
3.3.1 Free surface	19
3.3.2 Kelvin Wake	19
3.3.3 Prism Layers	19
3.4 Selection of CFD Numerical Parameters	20
3.5 Selection of DEM/Lagrangian Numerical Parameters	22
3.5.1 Material properties	22
3.5.2 Ice floe Geometry and Size Distribution	22
3.5.3 Ice Floe Field Generation	23
3.5.4 First Algorithm: Random Placement	24
3.5.5 Second Algorithm: Progressive Growth	24
3.5.6 Injection of the ice floes	26
3.6 Sensitivity Test on DEM parameters	27
3.6.1 Setup of the Sensitivity Test	27
3.6.2 Spring Stiffness Constant K	28
3.6.3 Time Scale	29

3.6.4	CFD Time Step	31
3.6.5	Conclusion	32
4	Results	33
4.1	Added Ice Resistance	33
4.1.1	Results and comparison with experimental results	33
4.2	Ice Floe Behavior upon Contact with the Ship	37
4.2.1	Rotating, Submerging and Sliding	37
4.2.2	Non-Realistic Behavior at Low Velocity	38
4.2.3	Non-Realistic Behavior at High Concentration	38
4.3	Impact on the ship	39
5	Additional work: particle clumps model investigation.	40
6	Conclusion	42
7	Future Work	43
A	Code	A-1
A.1	Ice Floe field random placement algorithm.	A-1
A.2	Ice Floe field progressive growth algorithm.	A-11

Nomenclature

B, L_{pp}, T	Ship breadth / length between perp. / draft	m
S	Wetted surface area of ship	m ²
ρ	Fluid density (water, air)	kg m ⁻³
μ	Fluid dynamic viscosity (water, air)	Pa s
g	Gravitational acceleration	m s ⁻²
$\bar{\mathbf{u}}$	Time-averaged fluid velocity	m s ⁻¹
$\bar{p}$	Time-averaged pressure	Pa
$\bar{\boldsymbol{\tau}}$	Viscous stress tensor	Pa
μ_t	Turbulent (eddy) viscosity	Pa s
k, ω	Turbulent kinetic energy / dissipation rate	—
α_w	Water volume fraction	—
y^+	Wall distance	—
F_r, Re	Froude number / Reynolds number	—
u_τ, τ_w	Friction velocity / wall shear stress	m s ⁻¹ , Pa
$T_{prism}, \Delta y$	Prism layer thickness / first cell height	m
R, t_{ice}	Ice floe radius / thickness	m
N, c_{ice}	Number of floes / ice concentration	—
S_{ice}	Surface area of the ice domain	m ²
ρ_{ice}	Ice density	kg m ⁻³
E_{ice}, ν_{ice}	Young's modulus / Poisson's ratio of ice	—
μ_{lg}, σ_{lg}	Log-normal distribution parameters	—
A_p, V_p	Projected area / volume of particle	—
m_p, M_{eq}	Particle mass / equivalent mass	kg
$\mathbf{v}_p, \mathbf{v}_s$	Particle velocity / slip velocity	m s ⁻¹
$\boldsymbol{\omega}_p, \boldsymbol{\omega}_{rel}$	Particle / relative angular velocity	rad s ⁻¹
I_p, I_{eff}, A_θ	Particle / effective / added moment of inertia	kg m ²
S_x, S_y	Moment-of-inertia scaling factors	—
$C_D, C_{D,\theta}, C_{vm}$	Drag / rotational drag / virtual mass coeff.	—
$\mathbf{F}_D, \mathbf{F}_p, \mathbf{F}_{vm}, \mathbf{F}_g$	Drag / pressure / virtual-mass / gravity force	N
$\mathbf{M}_b$	Hydrodynamic drag torque	N m
$\mathbf{F}_c$	Contact force	N
F_n, F_t	Normal / tangential contact force component	N
$\mathbf{M}_c$	Contact torque	N m
$K(K_n, K_t)$	Spring stiffness (normal / tangential)	N m ⁻¹
N_n, N_t	Normal / tangential damping coefficient	N s m ⁻¹
η_n, η_t	Normal / tangential damping ratio	—
C_{rest}, C_{fs}	Restitution / static friction coefficient	—
d_n, d_t	Normal overlap / tangential displacement	m
$\delta_{ij}, \delta_{max}$	Floe overlap / max. admissible overlap	m
R_{ice}, R_{OW}, R_T	Ice / open-water / total resistance	N
$F_{r,P}$	Ice (pack ice) Froude number	—
C_P	Pack ice resistance coefficient	—
C_1, C_2	Empirical fitting coefficients (Guo formula)	—
n	Ice concentration exponent	—
F_{RMS}	RMS of DEM contact force on hull	N
T_{avg}	Averaging period for RMS computation	s
F_{DEM}	DEM force on ship hull	N
$I(t)$	Cumulative DEM impulse	N s
t, dt, dt_{DEM}	Time / CFD time step / DEM time step	s
$t_{contact}, t_{sim}$	Duration of ice-floe contact / total simulation time	s
t_{scale}	DEM time scale factor	—
τ_1	Particle contact time scale	s

1 Introduction

1.1 Research motivation

Climate change is reshaping the prospects for navigation in the regions near the poles. The changing conditions in these parts of the world are driving the various actors of the maritime industry to develop methods for navigating efficiently and safely in these waters.

The opening of these new waterways offers opportunities for transit shipping, for the exploitation of resources such as oil, gas, minerals and fisheries [1], and, in the near future, for routing container ships from China to Europe through the Arctic [2]. This new route for container ships could drastically reduce the transit time — from roughly 28 to 18 days — thereby substantially lowering operating costs, while also helping to avoid the geopolitically unstable regions that disrupt world trade [3].

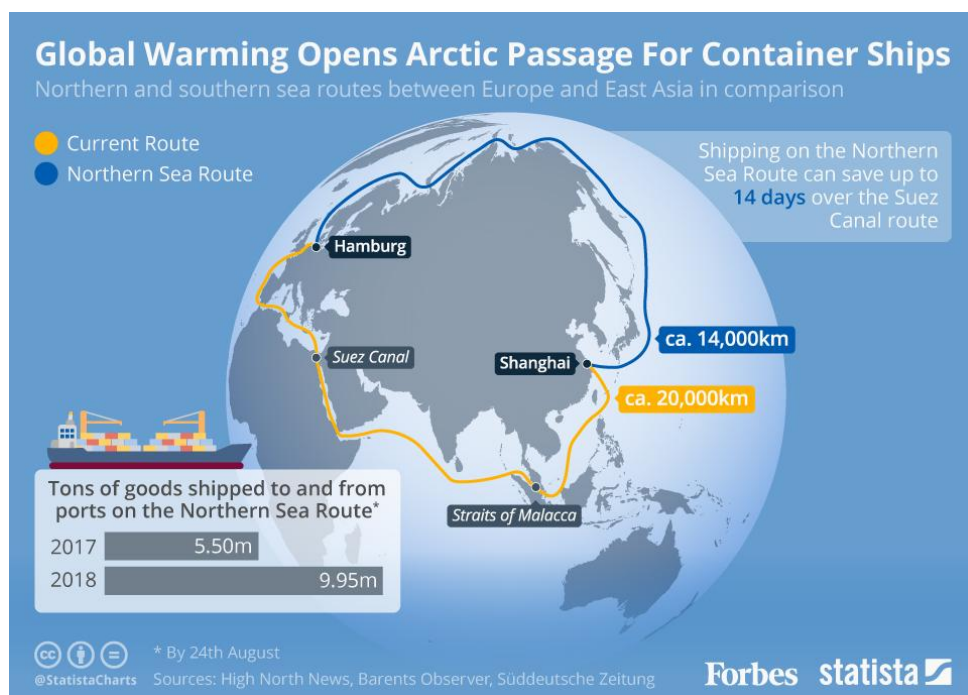


Figure 1: Northern and Southern sea routes between Europe and East Asia in comparison [4]

The efficient exploitation of these waterways relies on understanding the impact of the ice present at the free surface on ships, both structurally and in terms of the resistance to advance, relative to the more classical clean-water case. The free surface can be covered by ice in various forms — pack ice, ice floes or brash-ice channels, for example — and each condition imposes different loads and resistances on the ship. A poor understanding of these loads can lead to premature wear or even to the risk of severe damage (Figure 2).



Figure 2: Bulbous bow damaged by ice during navigation [5]

The case of interest here is that of ice floes, as these are expected to become the form of ice most frequently encountered by ships in the future, for both Arctic and Antarctic shipping [6]. Numerous studies, both experimental and numerical, have been published in recent years on this topic [7, 8, 9, 10], underlining the importance of the ice-added resistance acting on the ship. Such experimental campaigns are, however, extremely expensive, and are therefore not the most practical means of providing easy, low-cost predictions of the resistance of a ship navigating in ice floes.

A solution to this problem can be found in numerical simulation. Indeed, whereas setting up a towing-tank test can cost several thousand euros, a CFD simulation can prove considerably less expensive, which motivates the numerical approach adopted in this work.



Figure 3: Example of a ship navigating in ice floes [11]

1.2 Studies of ship–ice interaction

The interaction between a ship and broken ice has been investigated both experimentally and numerically.

Model-scale testing in ice tanks has long been the primary tool for assessing ship performance in ice. Because natural sea ice is difficult to scale correctly, broken-ice conditions are usually reproduced in towing tanks with synthetic ice. Guo et al. [7] tested a KCS model advancing through a synthetic pack-ice channel and measured the total resistance together with the heave and pitch motions over a range of speeds and ice concentrations, observing that the ratio of the pack-ice to the open-water resistance increases with concentration and decreases with speed. Luo et al. [8] examined the resistance and the motion-attitude variations produced by the combined ship–wave–ice interaction in the marginal ice zone, while Kim et al. [9] compared the resistance of an icebreaking cargo vessel for different waterline angles, showing that the bow geometry strongly affects the ice resistance. These experiments characterise how the ice floes increase the resistance and alter the motions of the hull, but they are costly, time-consuming and, in broken ice, of limited repeatability, since the random arrangement of the floes changes continuously during a run.

Numerical simulation offers a cheaper and more detailed alternative. Finite-element approaches based on explicit dynamics have been used to compute the ice loads, for example with LS-DYNA for an icebreaking cargo vessel in pack ice [12] and with a coupled Eulerian–Lagrangian formulation for ice impacts on hulls in broken-ice fields [10]. Guo et al. [13] also studied the KCS model through numerical simulation using the LS-DYNA software under the same conditions as the experimental study.

More recently, the coupling of Computational Fluid Dynamics (CFD) with the Discrete Element Method (DEM) has become the leading approach for broken-ice resistance, as it resolves both the free-surface flow and the discrete ship–ice and ice–ice collisions. Vroegrijk [14] first validated such a model in STAR-CCM+ against measured data. Huang et al. [15] studied wave interaction with a single circular floe and subsequently developed a CFD–DEM model that incorporated the ship-generated waves and analysed the dependence of the ice-floe resistance on ship speed, ice concentration, ice thickness and floe diameter for the KCS [16]. The same approach has been applied to an ice-strengthened bulk carrier in a brash-ice channel [17] and to full-scale ship–ice interaction under FSICR conditions [18], in good agreement with ice-tank tests.

1.3 Sea-ice characterisation and ice-ice interaction

Beyond the ship itself, the ice has been studied independently, since a realistic description of the floe field is essential for any resistance prediction. Two aspects are particularly relevant here: the geometry and properties of the individual floes, and the way the floes interact with one another.

The floes in a broken field are not uniform in size. Tuovinen [19] showed that the size of the ice blocks in a broken channel follows a log-normal distribution, which is now commonly used to generate representative ice fields in numerical models. The mechanical properties of the ice matter as well: Planque et al. [20] measured the Young’s modulus of ice and expressed it as a function of its density, providing input values for the contact models used in the simulations.

Because broken sea ice consists of many discrete bodies that collide, rotate and pack together, its behavior is well described by the Discrete Element Method (DEM), originally introduced by Cundall and Strack for granular assemblies [21]. Hopkins and Tuhkuri [22] used a DEM model to study the compression and ridging of floating ice fields, capturing the contact forces between individual floes, and Damsgaard et al. [23] showed that a particle-based DEM reproduces

the large-scale dynamics of sea ice more faithfully than continuum rheologies. These studies confirm that a discrete description is the natural way to represent the ice floes and their mutual interactions, and they underpin the modelling choices adopted in the present work.

1.4 Objectives and scope

The resistance applied to a ship navigating through ice comprises two components: the hydrodynamic resistance, and the resistance induced by contact with the ice floes. Capturing both aspects simultaneously motivates the coupling of a Computational Fluid Dynamics (CFD) solver with a Discrete Element Method (DEM) solver, which handles the fluid and the discrete ice–hull interactions respectively. The commercial software Simcenter STAR-CCM+ developed by Siemens provides an integrated CFD–DEM framework and is adopted in the present work.

The objective of this thesis is therefore to develop a numerical methodology, based on this CFD–DEM coupling, to investigate the added resistance of a ship advancing through an ice-floe field. The fluid phases are modelled with the RANS equations closed by the k – ω SST turbulence model [24], and the air–water interface is captured with the VOF method [25]. The ice floes are represented as discrete cylindrical Lagrangian particles, allowing hull–floe and floe–floe collisions to be resolved explicitly. The implementation is carried out on the KCS hull, which is widely used as a reference for numerical resistance studies [26], so that the computed ice resistance can be validated against the experimental data of Guo et al. [7]. Particular attention is paid to the generation of realistic ice-floe fields with prescribed concentrations, to the calibration of the DEM contact parameters, and to the known limitations of the CFD–DEM coupling regarding the hydrodynamic surface forces acting on the floes, which are addressed through corrections to the moment of inertia and the drag torque [27, 28]. The floe size distribution and material properties are selected following published data [19, 20].

2 Numerical Methods

This chapter describes the numerical model and its setup.

2.1 Fluid Solver

2.1.1 Navier–Stokes Equations

The fluid motion is governed by the Navier–Stokes equations. In this work, the Reynolds-averaged Navier–Stokes (RANS) equations are solved for an incompressible Newtonian fluid:

$$\nabla \cdot \bar{\mathbf{u}} = 0 \quad (1)$$

$$\frac{\partial(\rho_w \bar{\mathbf{u}})}{\partial t} + \nabla \cdot (\rho_w \bar{\mathbf{u}} \bar{\mathbf{u}}) = -\nabla \bar{p} + \nabla \cdot (\bar{\boldsymbol{\tau}} - \mathbf{R}) + \rho_w \mathbf{g} \quad (2)$$

where $\mathbf{u} = \bar{\mathbf{u}} + \mathbf{u}'$ is the instantaneous fluid velocity, $\bar{\mathbf{u}}$ its time-averaged component, $\mathbf{u}'$ the fluctuation, ρ the density, $\bar{p}$ the time-averaged pressure, $\bar{\boldsymbol{\tau}} = \mu [\nabla \bar{\mathbf{u}} + (\nabla \bar{\mathbf{u}})^T]$ the time-averaged viscous stress tensor, μ the dynamic viscosity, $\mathbf{g}$ the gravitational acceleration, and $\mathbf{R} = \rho \overline{\mathbf{u}' \mathbf{u}'}$ the Reynolds stress tensor [29].

2.1.2 Turbulence Modelling

Ship motions and wave–structure interactions can induce turbulent flow, so turbulence effects must be accounted for in the Navier–Stokes equations to obtain accurate results. Averaging the nonlinear convective term in the RANS equations introduces the additional Reynolds stress tensor $\mathbf{R}$, which increases the number of unknowns. A turbulence model is therefore required to close the system. The two most widely used turbulence models are the k – ε and k – ω formulations, each suited to different flow regimes [24]. In the present work, the k – ω SST model is selected, as it combines the robustness of the k – ε model in the far field with the accuracy of the k – ω model in the near-wall region. The principle is presented below using the k – ω model.

Using the Boussinesq approximation, the Reynolds stress tensor is modelled as

$$\mathbf{R} = -\mu_t [\nabla \bar{\mathbf{u}} + (\nabla \bar{\mathbf{u}})^T] + \frac{2}{3} \rho k \mathbf{I} \quad (3)$$

where μ_t is the eddy viscosity and k the turbulent kinetic energy, defined as

$$k = \frac{1}{2} \overline{u'_i u'_i} = \frac{1}{2} (\overline{u'^2} + \overline{v'^2} + \overline{w'^2}). \quad (4)$$

In the k – ω model, the eddy viscosity is expressed as a function of k and the specific dissipation rate ω :

$$\mu_t = \rho \frac{k}{\omega} \quad (5)$$

The variables k and ω are obtained by solving their respective transport equations:

$$\frac{\partial(\rho k)}{\partial t} + \nabla \cdot (\rho \bar{\mathbf{u}} k) = \nabla \cdot \left[\left(\mu + \frac{\mu_t}{\sigma_k} \right) \nabla k \right] + P_k - \beta^* \rho \omega k \quad (6)$$

$$\frac{\partial(\rho \omega)}{\partial t} + \nabla \cdot (\rho \bar{\mathbf{u}} \omega) = \nabla \cdot \left[\left(\mu + \frac{\mu_t}{\sigma_\omega} \right) \nabla \omega \right] + \gamma \frac{\omega}{k} P_k - \beta \rho \omega^2 \quad (7)$$

where P_k is the production term of turbulent kinetic energy, β^* , β , γ , σ_k and σ_ω are model constants [24].

2.1.3 Free-Surface Modelling

Free-surface problems involve two immiscible phases — water and air — and require a method to capture the interface between them. The Volume of Fluid (VOF) method is used here [25].

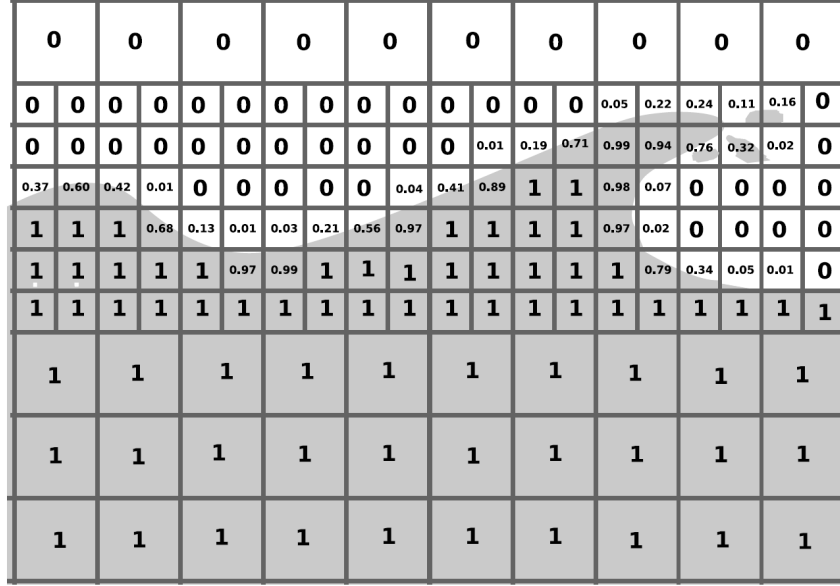


Figure 4: Illustration of free-surface reconstruction in the VOF method [30].

The VOF implementation in STAR-CCM+ predicts the distribution and motion of the interface between immiscible phases. The approach assumes that the mesh resolution is sufficient to resolve the position and shape of the interface. Phase distribution is described by the volume fraction α_w of water in each cell:

$$\alpha_w = \frac{V_{\text{water}}}{V} \quad (8)$$

where V_{water} is the volume of water in the cell and V is the total cell volume [31]. The value of α_w indicates the content of the cell:

- $\alpha_w = 0$: the cell contains only air;
- $\alpha_w = 1$: the cell contains only water;
- $\alpha_w \in (0, 1)$: the cell contains both phases (interface cell).

In the two-fluid VOF formulation, a single velocity and pressure field are solved across the domain. The material properties in interface cells are volume-averaged between the two phases:

$$\rho^{\text{cell}} = \alpha_w \rho_w + (1 - \alpha_w) \rho_a \quad (9)$$

$$\mu^{\text{cell}} = \alpha_w \mu_w + (1 - \alpha_w) \mu_a \quad (10)$$

The volume fraction is advected through the transport equation:

$$\frac{\partial \alpha_w}{\partial t} + \nabla \cdot (\bar{\mathbf{u}} \alpha_w) + \nabla \cdot [(\bar{\mathbf{u}}_w - \bar{\mathbf{u}}_a) \alpha_w (1 - \alpha_w)] = 0 \quad (11)$$

In this study, $\rho_w = 997.561 \text{ kg/m}^3$, $\mu_w = 8.8871 \times 10^{-4} \text{ Pa s}$, $\rho_a = 1.18415 \text{ kg/m}^3$, $\mu_a = 1.85508 \times 10^{-5} \text{ Pa s}$, and $g = 9.81 \text{ m/s}^2$.

2.2 Ice Floe Modelling

2.2.1 CFD–DEM Coupling

STAR-CCM+ provides the capability to couple the CFD solver with a DEM solver, allowing Lagrangian particles to be tracked within an Eulerian fluid flow. In the present work, the ice floes are modelled as DEM particles within the Eulerian fluid domain.

Two types of CFD–DEM coupling are available:

- **One-way coupling:** the DEM particles are influenced by the fluid flow, but they do not affect it in return.
- **Two-way coupling:** the DEM particles also exert forces on the fluid, modifying the flow field.

One-way coupling is selected here, as it provides sufficient accuracy at a substantially lower computational cost. Two-way coupling produces negligible differences in the predicted ship resistance while being significantly more expensive [17].

2.2.2 Lagrangian Description and DEM Formulation

In this simulation, the ice floes are modelled using a Lagrangian approach, in contrast to the surrounding fluids, which are described in an Eulerian framework. In an Eulerian description, flow variables are observed at fixed points in space, whereas the Lagrangian description follows individual entities along their trajectories [29]. This distinction is particularly relevant for CFD–DEM simulations, where the fluid phase is solved on a fixed computational grid while the discrete solid bodies are tracked individually.

A Lagrangian description is necessary for the ice floes because broken sea ice (such as ice floes) behaves as a collection of discrete bodies rather than as a continuous phase. Each floe undergoes its own translation, rotation, and contact interactions with neighbouring floes and with the ship hull. A particle-based Lagrangian approach is therefore better suited to sea ice modelling than continuum rheologies, since individual contact mechanics are more firmly established [23].

The method used to simulate each ice floe is the Discrete Element Method (DEM). The DEM is a numerical method designed to simulate the motion of many interacting discrete solid bodies. It extends the Lagrangian formulation to include inter-particle contact forces in the equations of motion [21]. STAR-CCM+ allows the user to define custom Lagrangian phases and phase-interaction models [32].

2.2.3 Governing Equations for the DEM Particles

The momentum equation for a DEM particle of mass m_p is:

$$m_p \frac{d\mathbf{v}_p}{dt} = \mathbf{F}_D + \mathbf{F}_p + \mathbf{F}_{vm} + \mathbf{F}_g + \mathbf{F}_c \quad (12)$$

where the right-hand-side terms are:

- $\mathbf{F}_D$, the drag force:

$$\mathbf{F}_D = \frac{1}{2} C_D \rho_w A_p |\mathbf{v}_s| \mathbf{v}_s \quad (13)$$

where C_D is the translational drag coefficient, A_p is the projected particle area, and $\mathbf{v}_s = \bar{\mathbf{u}} - \mathbf{v}_p$ is the slip velocity between the particle and the continuous phase.

- $\mathbf{F}_p$, the pressure-gradient force:

$$\mathbf{F}_p = -V_p \nabla \bar{p} \quad (14)$$

where V_p is the particle volume.

- $\mathbf{F}_{vm}$, the virtual mass force:

$$\mathbf{F}_{vm} = C_{vm} \rho_w V_p \left(\frac{D\bar{\mathbf{u}}}{Dt} - \frac{d\mathbf{v}_p}{dt} \right) \quad (15)$$

where C_{vm} is the virtual mass coefficient and D/Dt is the material derivative.

- $\mathbf{F}_g$, the gravitational force.
- $\mathbf{F}_c$, the contact force, defined below.

The angular-momentum equation for the particle is:

$$I_p \frac{d\boldsymbol{\omega}_p}{dt} = \mathbf{M}_b + \mathbf{M}_c \quad (16)$$

with:

- I_p , the particle moment of inertia;
- $\boldsymbol{\omega}_p$, the particle angular velocity;
- $\mathbf{M}_b$, the hydrodynamic drag torque:

$$\mathbf{M}_b = \frac{1}{2} \rho_w R^5 C_{D,\theta} |\boldsymbol{\omega}_{rel}| \boldsymbol{\omega}_{rel} \quad (17)$$

where R is the radius of the ice floe, $C_{D,\theta}$ is the rotational drag coefficient and $\boldsymbol{\omega}_{rel} = \frac{1}{2} \nabla \times \bar{\mathbf{u}} - \boldsymbol{\omega}_p$ is the relative angular velocity between the particle and the fluid;

- $\mathbf{M}_c$, the total moment from contact forces.

2.2.3.1 Contact force model. In the DEM model, the contact force $\mathbf{F}_c$ represents both particle–particle (ice–ice) and particle–boundary (ice–ship) interactions. At the contact point, the interaction is represented by a spring–dashpot oscillator.

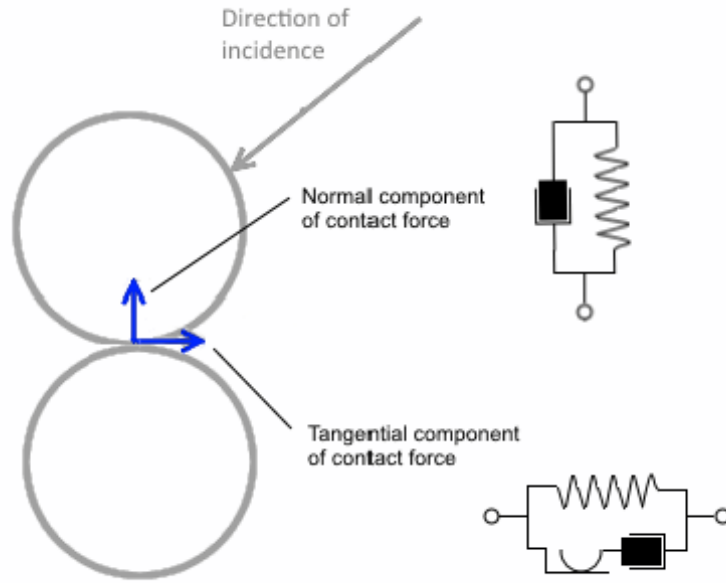


Figure 5: Spring–dashpot oscillator used to model the contact force between two particles. [33]

For the cylindrical ice floes considered here, only the Linear Spring contact model is available in STAR-CCM+ (the other models are made for other shapes, mainly sphere). The contact force between two cylinders A and B is decomposed into normal and tangential components:

$$\mathbf{F}_c = F_n \mathbf{n} + F_t \mathbf{t} \quad (18)$$

where $\mathbf{n}$ is the contact-point normal unit vector and $\mathbf{t}$ the tangential unit vector in the contact plane.

In the Linear Spring contact model, both for normal and tangential terms, the spring term accounts for the elastic repulsion generated by the overlap between the two particles, while the dashpot term provides viscous damping and dissipates part of the collision energy. The normal and tangential force components are [21]:

- Normal force:

$$F_n = -K_n d_n - N_n v_n \quad (19)$$

- Tangential force: $F_t = -K_t d_t - N_t v_t$ as long as $|K_t d_t| < |K_n d_n| C_{fs}$, where C_{fs} is the static-friction coefficient; otherwise:

$$F_t = \frac{|K_n d_n| C_{fs} d_t}{|d_t|} \quad (20)$$

Where d_n and d_t are overlaps in normal and tangential directions at the contact point, v_n and v_t are the normal and tangential velocity components of the relative particle surface velocity at the contact point.

The parameters K_n , K_t , N_n and N_t are defined as follows:

- K_n is the normal spring stiffness constant;
- K_t is the tangential spring stiffness constant;

- $N_n = 2\eta_n \sqrt{K_n M_{\text{eq}}}$, where η_n is the dimensionless normal damping ratio, obtained from the restitution coefficient C_{rest} via (21), and M_{eq} is the equivalent particle mass (22);
- $N_t = 2\eta_t \sqrt{K_n M_{\text{eq}}}$, where η_t is the dimensionless tangential damping ratio, obtained from the restitution coefficient C_{rest} via (21).

$$\eta = \frac{-\ln(C_{\text{rest}})}{\sqrt{\pi^2 + \ln^2(C_{\text{rest}})}} \quad (21)$$

$$M_{\text{eq}} = \frac{1}{\frac{1}{M_A} + \frac{1}{M_B}} \quad (22)$$

Two key parameters in the DEM contact model are the spring stiffness constants K_n and K_t , which convert the inter-particle overlap into a contact force. Their values are chosen based on a sensitivity test performed in Section 3.6.

The static friction coefficient C_{fs} and the restitution coefficient C_{rest} are purely physical parameters [28].

2.2.4 DEM solver parameters

To obtain the DEM particle trajectory, (12) is integrated over time using a characteristic DEM particle time scale. The particle time step depends on a contact time scale τ_1 and a user-defined scaling factor t_{scale} :

$$\tau_1 = 2\sqrt{\frac{m_p}{K}} \quad (23)$$

where m_p is the particle mass and K is the spring stiffness constant. In the present study, the normal and tangential spring stiffness constants are set equal, $K_n = K_t = K$, so that a single stiffness parameter governs both the contact model and the DEM time step. The DEM time step is then:

$$dt_{\text{DEM}} = t_{\text{scale}} \tau_1 \quad (24)$$

The spring stiffness constant K must therefore be selected carefully, as it governs both the physical accuracy and the computational cost of the simulation. A value that is too low results in excessive particle overlap, leading to artificially soft contact behavior and unreliable force predictions. Conversely, an excessively large value imposes a small DEM time step according to (23) and increases the computational cost. In practice, K should be chosen as the minimum value and t_{scale} as the maximum value for which the simulation results become independent of further stiffness increases. A sensitivity study was therefore conducted in this work to identify this convergence threshold, as discussed in Section 3.6.

2.2.5 Limitations of the CFD–DEM Model

The CFD–DEM coupling introduces modelling assumptions that can lead to non-physical behavior for individual ice floes. Two limitations are discussed below: the inability to enforce the correct equilibrium draft, and the simplified representation of hydrodynamic surface forces.

As expressed in (12) and (16), the hydrodynamic forces acting on a DEM particle are computed using simplified force models rather than by integrating the fluid stress over the actual particle surface. Since DEM particles are treated as rigid bodies subject to resultant forces and

torques at their centre of mass, spatially distributed surface loads — such as hydrostatic pressure gradients — cannot be represented in a physically consistent manner.

Regarding the equilibrium draft, this limitation is inherent to the DEM framework: the correct submergence depth of the floe, governed by the density ratio ρ_{ice}/ρ_w , cannot be enforced through the DEM force model alone. No straightforward correction is available within the present coupling strategy, and this represents an acknowledged source of modelling error.

2.2.5.1 Hydrodynamic Torque and Added Moment of Inertia. The hydrodynamic torque associated with added mass is not directly computed by the DEM model. However, its effect can be introduced implicitly by augmenting the particle moment of inertia. STAR-CCM+ provides a scaling factor for the moment of inertia of each particle about its principal axes, which can be exploited for this purpose. Since the added-mass torque is proportional to the angular acceleration of the particle, it is formally equivalent to an increase in rotational inertia. For a rotational degree of freedom, the added-mass torque is:

$$\mathbf{M}_a = -A_\theta \frac{d\boldsymbol{\omega}_p}{dt} \quad (25)$$

where A_θ is the added moment of inertia associated with rotational motion. This contribution is incorporated into the equation of motion by replacing the physical moment of inertia I_p with an effective value:

$$I_{\text{eff}} = I_p + A_\theta \quad (26)$$

The ice floe is assumed to be initially horizontal, with its symmetry axis vertical. Upon contact with the ship hull, it undergoes roll or pitch rotation about a horizontal axis. The corresponding added moments of inertia are equal by axial symmetry and given by [27]:

$$A_{\theta x} = A_{\theta y} = \frac{8}{45} \rho_w R^5 \quad (27)$$

The physical moments of inertia of a thin cylindrical floe about the horizontal axes are:

$$I_x = I_y = \frac{1}{12} m_p (3R^2 + t_{\text{ice}}^2) \quad (28)$$

where t_{ice} is the floe thickness and $m_p = \rho_{\text{ice}} \pi R^2 t_{\text{ice}}$ its mass. In the thin-floe limit $t_{\text{ice}} \ll R$, this simplifies to:

$$I_x = I_y \simeq \frac{1}{4} \rho_{\text{ice}} \pi t_{\text{ice}} R^4 \quad (29)$$

The ratio of added to physical moment of inertia is therefore:

$$\frac{A_{\theta x}}{I_x} = \frac{A_{\theta y}}{I_y} = \frac{32}{45\pi} \frac{\rho_w}{\rho_{\text{ice}}} \frac{R}{t_{\text{ice}}} \quad (30)$$

Accordingly, the moment of inertia scaling factor to be applied in STAR-CCM+ is:

$$S_x = S_y = \frac{I_p + A_\theta}{I_p} = 1 + \frac{32}{45\pi} \frac{\rho_w}{\rho_{\text{ice}}} \frac{R}{t_{\text{ice}}} \quad (31)$$

For typical ice and water densities ($\rho_{\text{ice}} \approx 900 \text{ kg/m}^3$, $\rho_w \approx 1025 \text{ kg/m}^3$) and the mean value of the floe geometry used in this study ($R = 0.1 \text{ m}$, $t_{\text{ice}} = 0.02 \text{ m}$), this gives $S_x = S_y \approx 2.25$.

2.2.5.2 Particle Drag Torque. The rotational drag coefficient $C_{D,\theta}$ used in the DEM torque model (17) must be determined for the cylindrical ice floes considered here. It is derived from Morison's quadratic drag equation applied to a thin disk in roll or pitch.

The elementary drag force acting on a surface element dA is:

$$dF_D = \frac{1}{2} \rho_w C_D |V_r| V_r dA \quad (32)$$

For a rigid body rotating with angular velocity $\dot{\theta}$ about the y -axis, a point at horizontal distance x from the rotation axis has a normal velocity:

$$V_r = x \dot{\theta} \quad (33)$$

Substituting into the Morison equation, the elementary drag force becomes:

$$dF_D = \frac{1}{2} \rho_w C_D \dot{\theta} |\dot{\theta}| x^2 dA \quad (34)$$

The corresponding elementary drag torque, with moment arm x , is:

$$dM_D = x dF_D = \frac{1}{2} \rho_w C_D \dot{\theta} |\dot{\theta}| x^3 dA \quad (35)$$

Integrating over the wetted half-disk $\mathcal{D}^+$ yields the total drag torque:

$$M_D = -\frac{1}{2} \rho_w C_D \dot{\theta} |\dot{\theta}| \int_{\mathcal{D}^+} x^3 dA \quad (36)$$

where the negative sign reflects the resisting nature of the drag torque, acting in the direction opposite to $\dot{\theta}$.

The half-disk $\mathcal{D}^+$ is decomposed into vertical strips parallel to the y -axis, as illustrated in Figure 6. Since the disk boundary satisfies $x^2 + y^2 \leq R^2$, the strip at a given x extends from:

$$-\sqrt{R^2 - x^2} \leq y \leq +\sqrt{R^2 - x^2} \quad (37)$$

so that the strip height and elementary area are:

$$h(x) = 2\sqrt{R^2 - x^2}, \quad dA = h(x) dx = 2\sqrt{R^2 - x^2} dx \quad (38)$$

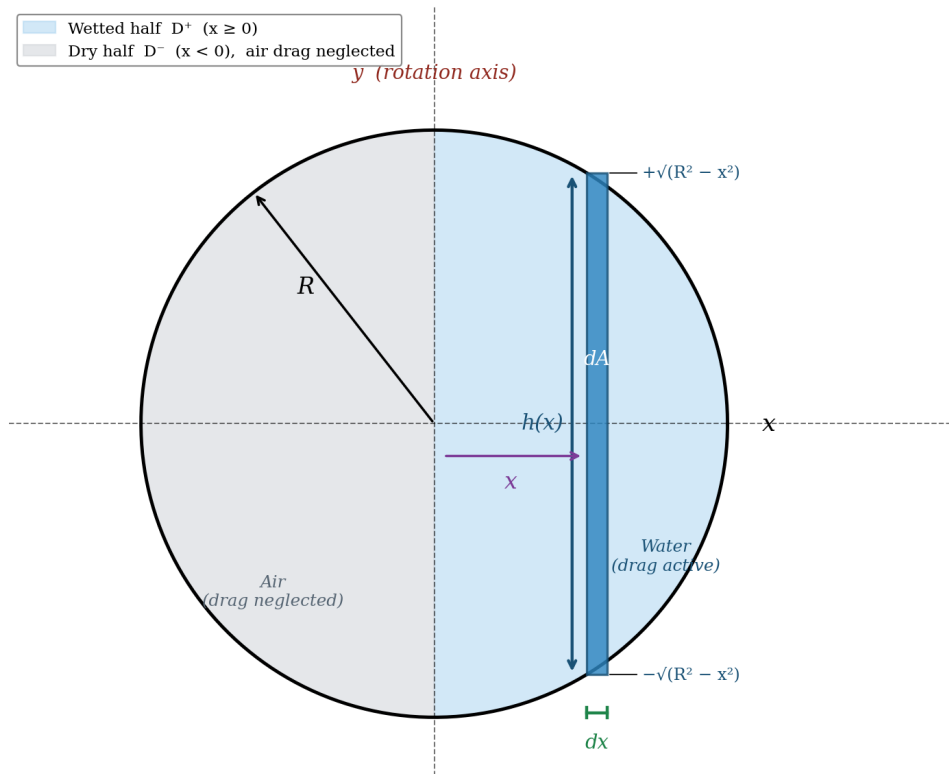


Figure 6: Vertical-strip decomposition of the wetted half-disk $\mathcal{D}^+$ used to evaluate the geometric integral of the drag torque.

The geometric integral in (36) becomes:

$$\mathcal{I} = \int_{\mathcal{D}^+} x^3 dA = 2 \int_0^R x^3 \sqrt{R^2 - x^2} dx \quad (39)$$

Applying the trigonometric substitution $x = R \sin \varphi$, $dx = R \cos \varphi d\varphi$, $\sqrt{R^2 - x^2} = R \cos \varphi$, with bounds $x = 0 \Rightarrow \varphi = 0$ and $x = R \Rightarrow \varphi = \pi/2$:

$$\mathcal{I} = 2 \int_0^{\pi/2} \underbrace{R^3 \sin^3 \varphi}_{x^3} \cdot \underbrace{R \cos \varphi}_{\sqrt{R^2 - x^2}} \cdot \underbrace{R \cos \varphi d\varphi}_{dx} = 2R^5 \int_0^{\pi/2} \sin^3 \varphi \cos^2 \varphi d\varphi \quad (40)$$

Using the identity $\sin^3 \varphi = \sin \varphi (1 - \cos^2 \varphi)$ and the substitution $u = \cos \varphi$:

$$\int_0^{\pi/2} \sin^3 \varphi \cos^2 \varphi d\varphi = \int_0^1 u^2 du - \int_0^1 u^4 du = \frac{1}{3} - \frac{1}{5} = \frac{2}{15} \quad (41)$$

The geometric integral therefore evaluates to:

$$\boxed{\mathcal{I} = \frac{4}{15} R^5} \quad (42)$$

Substituting back into (36), the drag torque on the ice floe at first contact is:

$$M_D = -\frac{1}{2} \rho_w \frac{4}{15} C_D R^5 \dot{\theta} \left| \dot{\theta} \right| \quad (43)$$

This expression is now compared with the STAR-CCM+ rotational drag torque introduced in (17):

$$\mathbf{M}_b = \frac{1}{2} \rho_w R^5 C_{D,\theta} |\boldsymbol{\omega}_{\text{rel}}| \boldsymbol{\omega}_{\text{rel}}$$

By identification of the two expressions, the rotational drag coefficient to be entered in STAR-CCM+ for the cylindrical ice floes considered here is:

$$C_{D,\theta} = \frac{4}{15} C_D \quad (44)$$

2.3 Summary of Numerical Parameters

This subsection summarises the numerical and physical parameters used in the CFD–DEM simulations of the present study. The values are gathered here for quick reference and reproducibility.

2.3.1 Fluid Properties

The two fluid phases (water and air) are modelled as incompressible Newtonian fluids with the properties listed in Table 1.

Property	Symbol	Value	Unit
Water density	ρ_w	997.561	kg/m ³
Water dynamic viscosity	μ_w	8.8871×10^{-4}	Pa s
Air density	ρ_a	1.18415	kg/m ³
Air dynamic viscosity	μ_a	1.85508×10^{-5}	Pa s
Gravitational acceleration	g	9.81	m/s ²

Table 1: Fluid properties used in the simulations.

2.3.2 Ice Floe Geometry and Properties

The ice floes are modelled as thin cylindrical disks. Their geometric and material properties are listed in Table 2.

Property	Symbol	Value	Unit
Ice density	ρ_{ice}	900	kg/m ³
Poisson’s ratio	ν	0.33	—
Young’s modulus	E_{ice}	9.0	GPa

Table 2: Ice floe geometry and physical properties.

2.3.3 DEM Hydrodynamic Coefficients

The hydrodynamic force and torque coefficients applied to the DEM particles are summarised in Table 3. The rotational drag coefficient $C_{D,\theta}$ corresponds to the value derived in (44) from the half-disk model.

Coefficient	Symbol	Value	Comment
Translational drag coefficient	C_D	1.2	From [28]
Virtual mass coefficient	C_{vm}	1.0	From [28]
Rotational drag coefficient	$C_{D,\theta}$	$\frac{4}{15} C_D = 0.32$	From (44)
Moment-of-inertia scaling	$S_x = S_y$	≈ 2.25	From (31)

Table 3: Hydrodynamic coefficients used in the DEM model.

2.3.4 DEM Contact Model Parameters

The parameters of the Linear Spring contact model are listed in Table 4. The normal and tangential spring stiffness constants are chosen from the sensitivity test (Section 3.6).

Parameter	Symbol	Value	Unit
Normal spring stiffness	K_n	4000000	N/m
Tangential spring stiffness	K_t	4000000	N/m
Restitution coefficient	C_{rest}	0.8098	—
Static friction coefficient (ice-ice)	C_{fs}	0.35	—
Static friction coefficient (ice-wall)	C_{fs}	0.035	—

Table 4: DEM contact model parameters.

2.3.5 Numerical Setup

The simulation control parameters (time step, turbulence model, solver schemes) are summarised in Table 5.

Setting	Value
Solver type	Segregated, implicit unsteady
Turbulence model	$k-\omega$ SST
Free-surface model	Volume of Fluid (VOF), High-Resolution Interface Capturing
CFD–DEM coupling	One-way
CFD Time step	$dt = 0.01$ s
Duration of ice-floe contact	$t_{contact} = 40$ s
Total simulation time	$t_{sim} = 80$ s
Inner iterations / step	5

Table 5: Numerical setup of the CFD–DEM simulation.

3 Computational Setup

3.1 KCS Geometry

A modern container ship model, the KRISO Container Ship (KCS), was used to validate the numerical model for navigation in ice floes. KCS is a typical container ship model widely employed in computational simulations, which makes it valuable for comparison with existing reference data [7].

The full-scale length of the hull is 230 m, and a scale ratio of 1:52.667 is applied in this study, resulting in the following characteristic parameters:

- Length between perpendiculars: $L_{pp} = 4.367$ m
- Breadth: $B = 0.611$ m
- Draft: $T = 0.20489$ m
- Wetted surface area: $S = 3.483$ m²

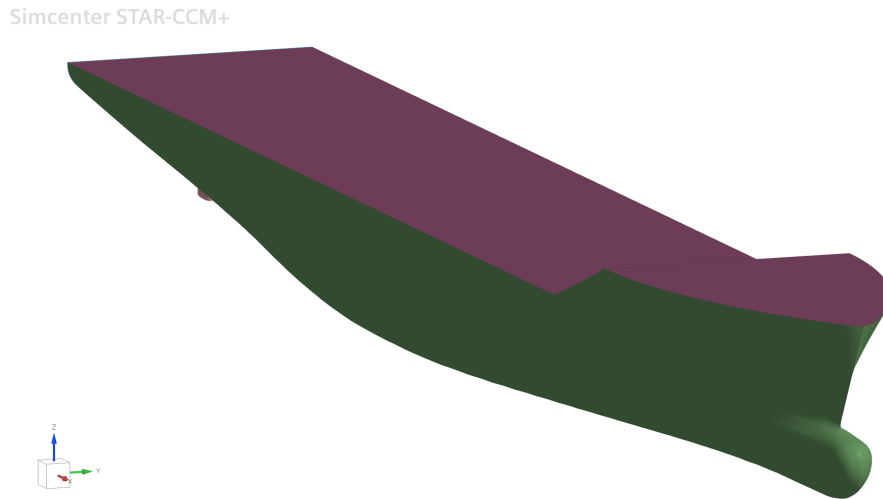


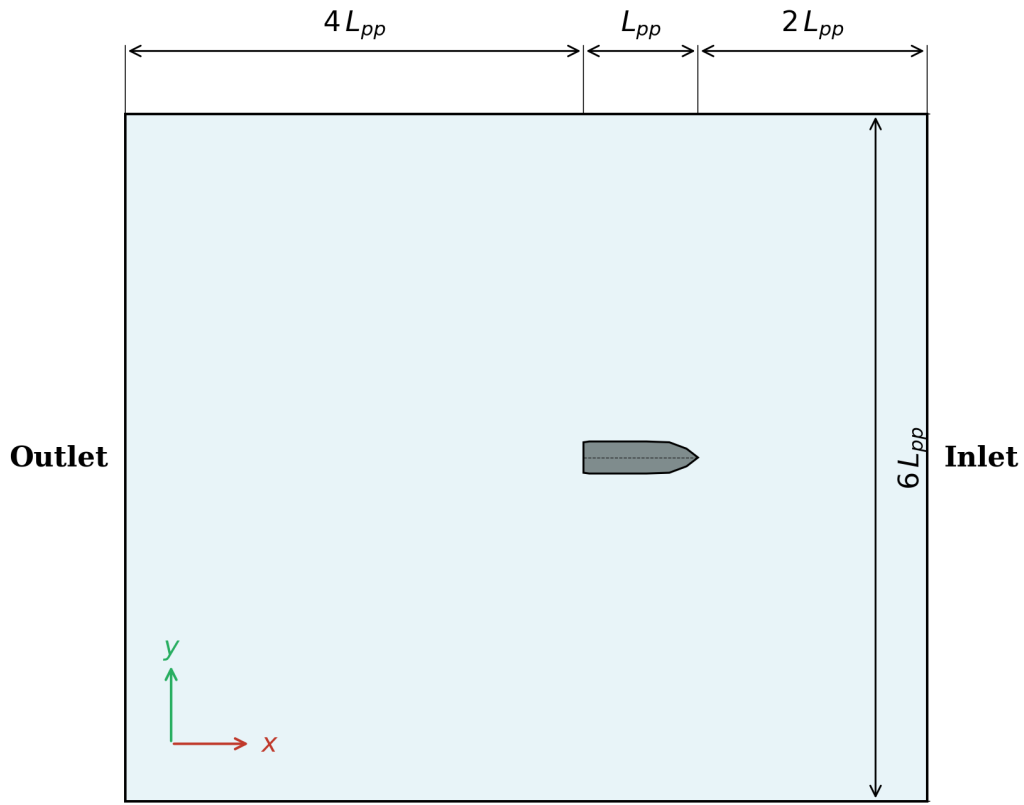
Figure 7: KCS geometry

3.2 Domain and Boundary Conditions

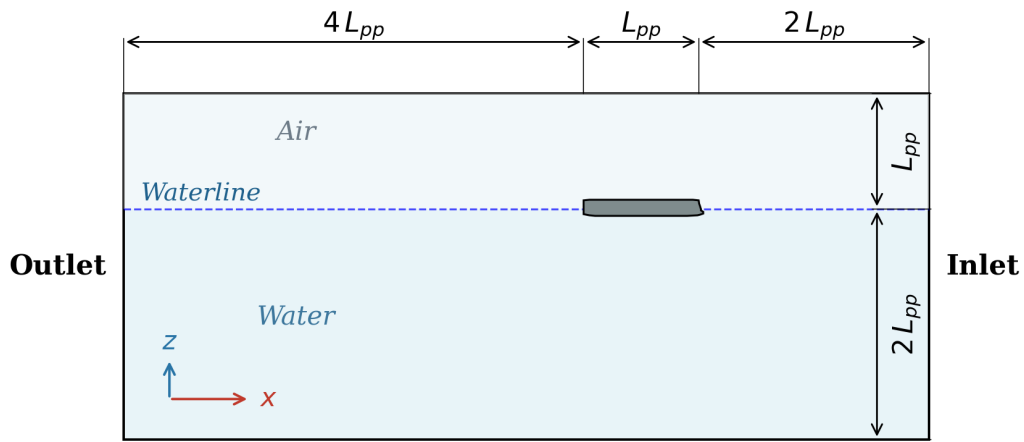
For the numerical simulation of the ship resistance in calm water, different approaches can be used. The ship can either be kept fixed while an incoming flow is imposed, or it can be allowed to move through an initially still fluid. In the present study of the KCS hull, the fixed-ship approach was selected, since it is simpler to implement from a numerical point of view. Moreover, for this type of resistance simulation, the expected differences between the two approaches are limited, as the motion is essentially steady and the acceleration effects remain small.

Following the guidelines of the International Towing Tank Conference (ITTC) [34], an open-ocean fluid domain was created, as illustrated in Figure 8. The computational domain is three-dimensional and defined in an earth-fixed Cartesian coordinate system Oxyz, where the (x,y) plane is parallel to the undisturbed free surface.

Unlike standard open-water CFD simulations, the full ship geometry must be included in the domain — rather than exploiting the longitudinal symmetry plane — since the random spatial distribution of ice floes breaks the port-starboard symmetry of the problem.



(a) Plan view (xy-plane)



(b) Profile view (xz-plane)

Figure 8: Computational domain of the KCS.

In this initial implementation, the ship is kept fixed with zero degrees of freedom (0 DOF). This simplification is justified at low advance speeds, where the vertical motion (heave) and pitch rotation induce only limited hydrodynamic effects, and allows a significant reduction in computational cost. The influence of ship motions on ice resistance will be investigated in future work.

Water and air are both assigned the same constant velocity ($U_{water} = U_{air}$) at the inlet of

the domain. Thus, a relative velocity exists between the ship and the water, corresponding to the real-life condition in which the ship navigates through calm water at a speed $U_{ship} = U_{water}$. In the following, the ship velocity will not be written explicitly, but rather expressed through the Froude number, defined as $F_r = \frac{U}{\sqrt{gL_{pp}}}$, where g is the gravitational acceleration and L_{pp} is the ship length between perpendiculars.

For the remaining domain boundaries, a fixed dynamic pressure is imposed at the outlet, symmetry plane conditions are applied on the side boundaries, and inlet conditions are prescribed at the top and bottom boundaries. The ship surface is modelled as a no-slip wall.

3.3 Mesh

A trimmed-cell mesh is generated using local refinement zones; a surface remesher is applied to refine the imported hull geometry. Prism layers are also included to model the boundary layer near the hull surface. Since a wall-function approach is used ($y^+ \approx 30$), an accurate representation of the boundary layer is essential, as it governs both the frictional resistance (wall shear stresses) and the pressure resistance (pressure distribution around the hull and flow separation).

The mesh base size is 0.0675m. To ensure an accurate resolution, several mesh refinement zones are required. An overview of the refinement zones is given below.

Table 6: Mesh refinement zones.

Refinement zone	Refinement direction	Cell size (m)	Percentage of base size
Free Surface	Z	0.0042	6.25%
Bulbous bow	X, Y, Z	0.0084	12.50%
Stern	X, Y, Z	0.0084	12.50%
Hull	X, Y	0.0338	50.00%
Hull	Z	0.0169	25.00%
Kelvin Wake	X, Y	0.0338	50.00%

3.3.1 Free surface

In any ship resistance case, it is very important to accurately resolve the air-water interface, which requires a refined free-surface mesh. In addition, for ice floes, it is recommended to use at least 4 cells per ice thickness, which corresponds to a cell size of 5 mm for an ice thickness of 20 mm.[15]

3.3.2 Kelvin Wake

These refinement zones are quite general for a mesh used in a ship resistance study in deep water. One difference is the Kelvin wake region, which is more refined to better track the particle motion[16].

3.3.3 Prism Layers

In the present study, to optimize computational efficiency, the viscous sublayer is not directly resolved. Instead, the mesh is designed such that the first cell height corresponds to the log-law region, targeting a value $y^+ \geq 30$.

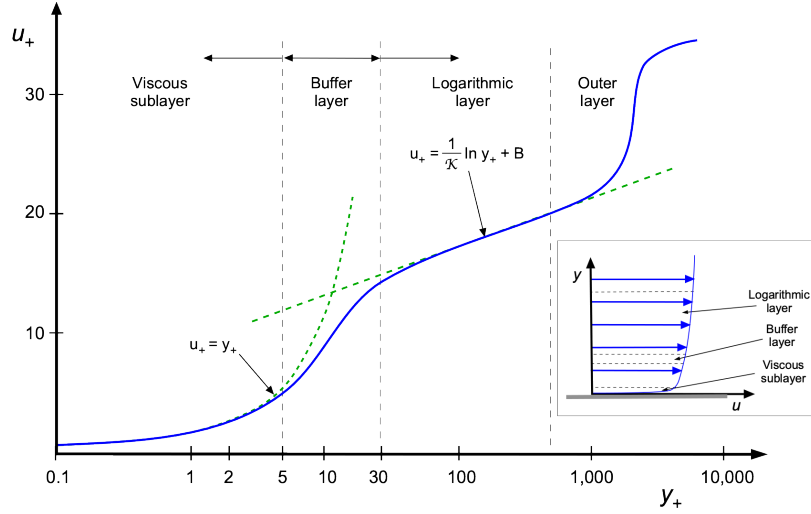


Figure 9: Turbulence regions in the boundary layer [35]

To define the near-wall mesh, the thickness of the inflation layer (prism layers) is determined based on the estimated boundary layer thickness, δ . In this study, the prism layer is designed to cover approximately 20% of the theoretical boundary layer thickness to ensure a smooth transition between the structured near-wall mesh and the unstructured far-field mesh. This value has been verified in CFD by assuring the boundary layer is within the prism layer region.

The first cell height, Δy , is calculated to target a dimensionless wall distance of $y^+ \geq 30$, consistent with the use of wall functions. This height is estimated using the following relations:

$$\Delta y = \frac{y^+ \mu}{\rho u_\tau} \quad (45)$$

where the friction velocity u_τ is derived from the wall shear stress τ_w :

$$u_\tau = \sqrt{\frac{\tau_w}{\rho}} = U_\infty \sqrt{\frac{C_f}{2}} \quad (46)$$

The skin friction coefficient C_f is estimated using the ITTC-1957 correlation line for ship-flow applications [34]:

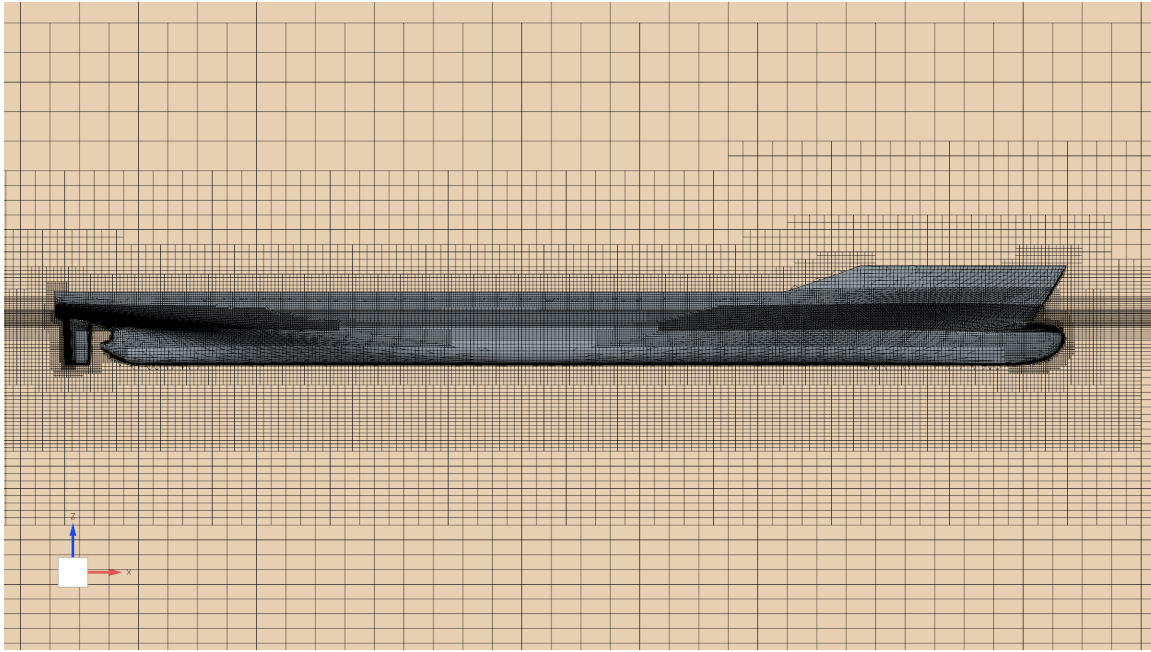
$$C_f = \frac{0.075}{(\log_{10} Re - 2)^2} \quad (47)$$

3.4 Selection of CFD Numerical Parameters

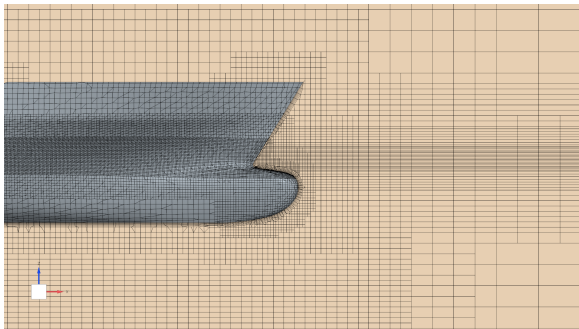
For all the simulations, the time step has been set to $dt = 0.01$ s.

The convergence within each time step was controlled by the number of **inner iterations**. Preliminary tests were conducted to determine the optimal balance between stability and computational cost:

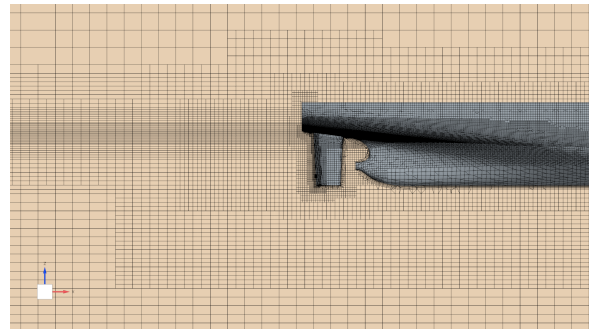
- Five inner iterations were found to provide a robust compromise, ensuring stable convergence of the residuals at each step.
- Increasing this number beyond five yielded no significant improvement in solution accuracy, whereas lower values led to unstable behavior and poor pressure-velocity coupling.



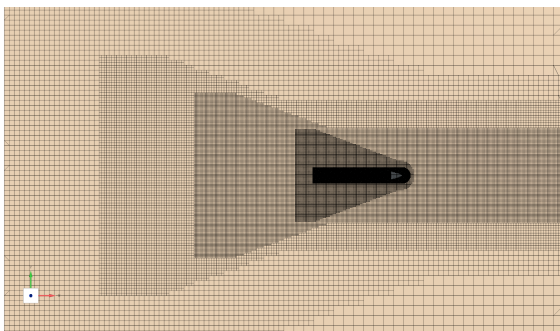
(a) Side view of the mesh around the hull.



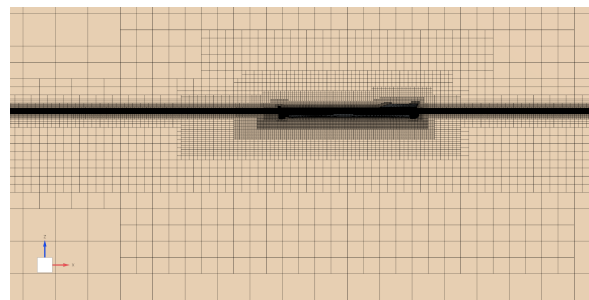
(b) Detailed view of the mesh at the bow.



(c) Detailed view of the mesh at the stern.



(d) View at the free surface.



(e) View on the symmetry plane.

Figure 10: Numerical mesh of the computational domain and local refinements around the geometry.

3.5 Selection of DEM/Lagrangian Numerical Parameters

3.5.1 Material properties

The ice floes are defined as solid, cylindrical DEM particles with constant material properties. The ice density is set to $\rho_{\text{ice}} = 900.0 \text{ kg/m}^3$, while the Poisson's ratio and Young's modulus are set to $\nu_{\text{ice}} = 0.33$ and $E_{\text{ice}} = 9.0 \text{ GPa}$, respectively [20].

For the contact model, different friction coefficients are assigned depending on the type of interaction. The ice–ice friction coefficient is set to 0.35, while the ice–ship friction coefficient is set to 0.035 [7]. The same restitution coefficient is used for both the normal and tangential directions, and for both ice–ice and ice–steel contacts, with $C_{\text{rest}} = 0.8098$.

3.5.2 Ice floe Geometry and Size Distribution

The geometry selected to represent the ice floes in the simulation is a cylinder. Although ice floes are modelled as square prisms in the experimental reference cases, the cylindrical shape was preferred for two reasons: the influence of floe geometry on the added resistance has been shown to be limited [16], and the cylindrical shape considerably simplifies the implementation. To ensure a consistent size equivalence between the experimental and numerical ice floes, both geometries are matched by cross-sectional area: for a given floe size probability, a simulated cylindrical floe and its experimental square counterpart share the same surface area, and consequently the same mass.

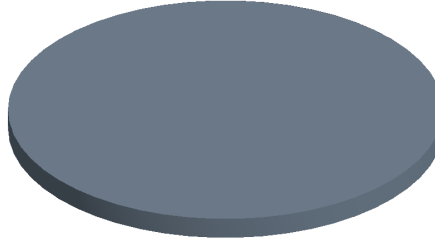


Figure 11: Example of ice floe: Diameter = $0.4m$, thickness = $0.02m$

The ice floe diameters are assumed to follow a log-normal distribution [19].

$$f_X(x) = \frac{1}{x\sigma_{\text{lg}}\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu_{\text{lg}})^2}{2\sigma_{\text{lg}}^2}\right) \quad (48)$$

For the KCS case, the parameters of the log-normal distribution were selected to match the experimental data reported in [7].

The resulting diameter probability density function is shown in Figure 12.

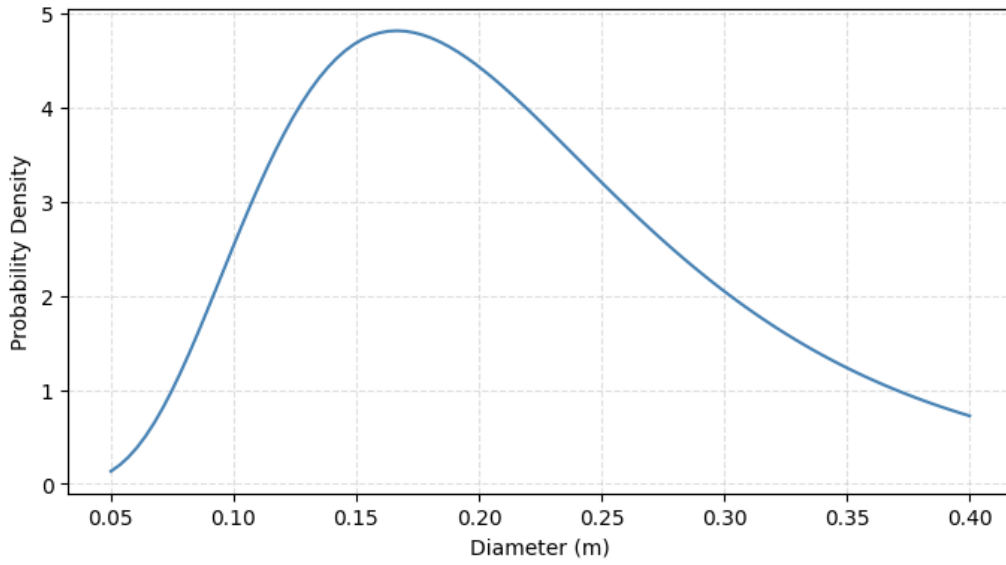


Figure 12: Ice floes diameter distribution: log-normal, $\mu_{lg} = -1.59$, $\sigma_{lg} = 0.45$

The ice floe diameters range from 5 cm to 40 cm with a constant thickness $t_{ice} = 2$ cm.

3.5.3 Ice Floe Field Generation

The number of ice floes required to reach a prescribed ice concentration is determined from the cumulative ice-covered area. Ice floes are generated with random diameters drawn from the selected log-normal distribution until the total ice area exceeds:

$$c_{ice} A_{domain},$$

where c_{ice} is the target ice concentration and A_{domain} is the surface area of the ice domain. Two algorithms were written to generate the ice floe field.

3.5.4 First Algorithm: Random Placement

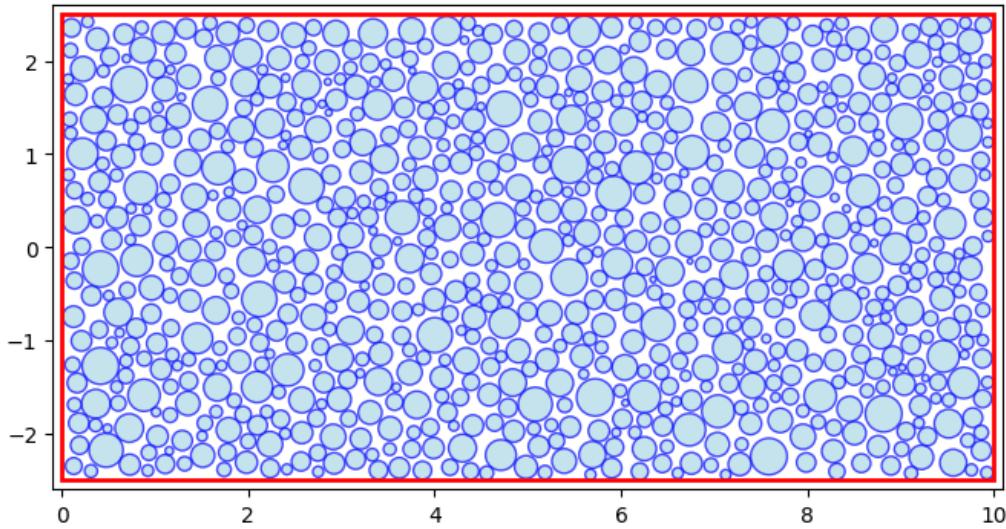


Figure 13: Example of ice floes field generation with the first algorithm, $c_{\text{ice}} = 60\%$

A first simple algorithm was developed. In this approach, the floes are sorted in decreasing order of diameter and then placed randomly inside the domain while avoiding overlap (See Figure 13). This method provides satisfactory results for ice concentrations below or equal to 60%. However, for higher concentrations, the algorithm becomes less efficient, since the available open-water space decreases significantly and the random placement of all floes becomes increasingly difficult. The code is available in Appendix A.1

3.5.5 Second Algorithm: Progressive Growth

A second algorithm was implemented in order to generate ice floe fields with high ice concentration. This method is based on a progressive-growth strategy. First, the final floe diameters are generated from a truncated log-normal distribution, bounded by a minimum and maximum diameter. The number of floes is chosen such that the target ice concentration is reached, where the concentration is defined as

$$c_{\text{ice}} = \frac{\sum_{i=1}^N \pi r_i^2}{S_{\text{ice}}},$$

with r_i the radius of floe i , N the total number of floes, and S_{ice} the surface area of the ice domain.

Instead of placing the floes directly at their final size, all floes are initially inserted with a reduced radius (45% of their final radius). This substantially reduces the probability of initial overlap and makes the packing process more robust. The initial positions are chosen randomly inside the domain using an empty-space criterion: for each floe, several candidate positions are tested, and the position with the largest clearance from the already placed floes is selected.

After this initial placement, the floes are progressively enlarged until their final size is reached. At each growth step, the radius of every floe is multiplied by a scale factor that is gradually increased from the initial value to 1. After each increase in size, the algorithm checks the overlap between all pairs of floes. For two floes i and j , the overlap is defined as

$$\delta_{ij} = r_i + r_j - \|\mathbf{x}_i - \mathbf{x}_j\|,$$

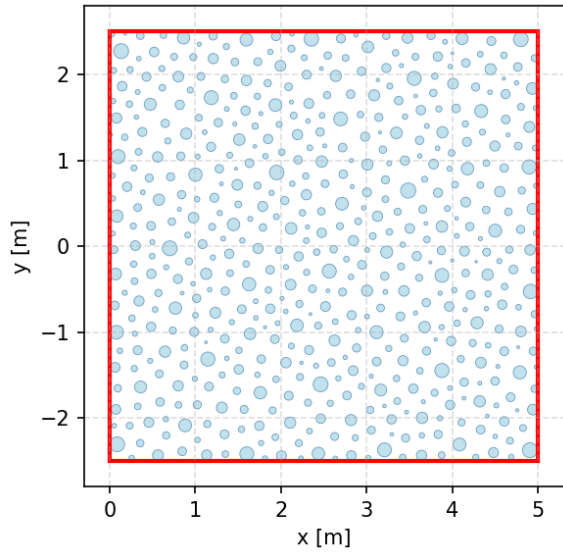
where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the floe center positions. An overlap occurs when $\delta_{ij} > 0$. The global overlap penalty is then computed as

$$P = \sum_{\delta_{ij} > 0} \delta_{ij}^2.$$

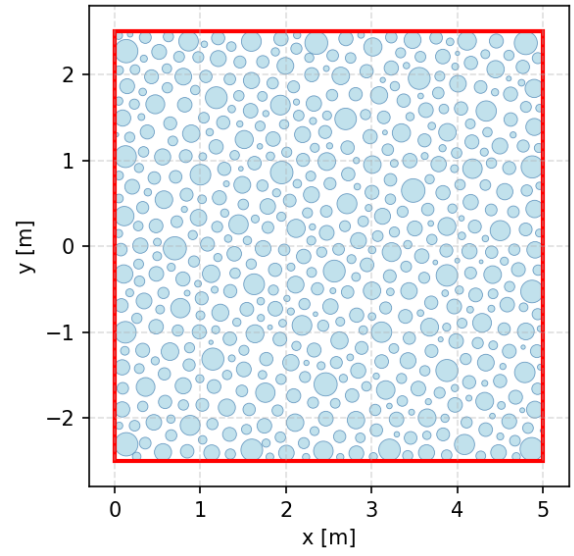
If overlaps are detected, a relaxation procedure is applied. Overlapping floes are pushed apart in the direction connecting their centers. Smaller floes are allowed to move more than larger floes, which improves the stability of the packing. If the overlap penalty remains too large after relaxation, the growth step is rejected and reduced. In difficult configurations, some of the smallest problematic floes are relocated to better positions in order to escape local blocked arrangements.

Once the final scale is reached, a final relaxation step is performed. If only very small residual overlaps remain, a slight uniform reduction of all floe radius is applied to remove them completely while keeping the ice concentration almost unchanged. This correction is negligible compared with the floe size, since it is only applied when the maximum overlap is below a prescribed tolerance.

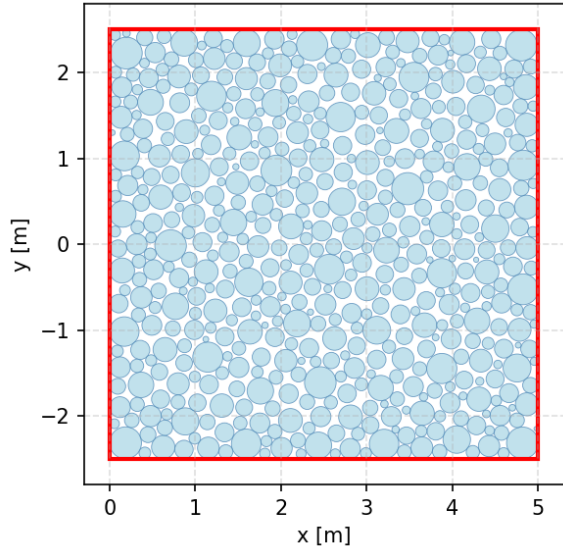
This progressive-growth algorithm is efficient for high-concentration ice fields. In the present case, it allows the generation of an ice floe field with a target concentration of $c_{\text{ice}} = 80\%$ in a domain of area $S_{\text{ice}} = 25 \text{ m}^2$. The different stages of the growth process are shown in [Figure 14](#). The code is available in [Appendix A.2](#).



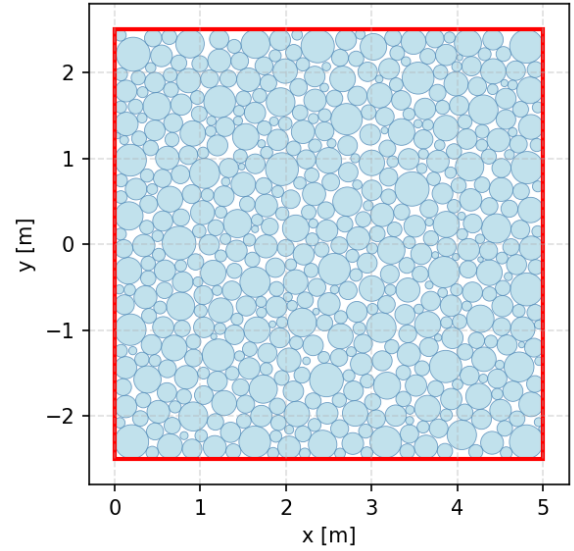
(a) Progressive growth $n^{\circ}0$, scale = 0.440, $c_{ice} = 15.5\%$



(b) Progressive growth $n^{\circ}5$, scale = 0.686, $c_{ice} = 37.7\%$



(c) Progressive growth $n^{\circ}10$, scale = 0.936, $c_{ice} = 70.2\%$



(d) Final progressive-growth ($n^{\circ}20$) ice floe field, $c_{ice} = 80.1\%$

Figure 14: Example of generation of ice floe field with Algorithm II, $c_{ice} = 80.1\%$, $S_{ice} = 25 \text{ m}^2$, 538 floes.

3.5.6 Injection of the ice floes

The introduction of ice floes into the computational domain is performed using an array injection strategy. The simulation is first run without ice floes in order to allow the fluid solution to reach a steady-state condition. Once this state is achieved, an array of ice floes is injected into the domain near the inlet. Each floe is initialized with a velocity equal to the fluid velocity. When the first array has travelled a distance L_{array} , a second array is injected so that it follows the previous one. This process is repeated to ensure a continuous inflow of ice floes into the computational domain.

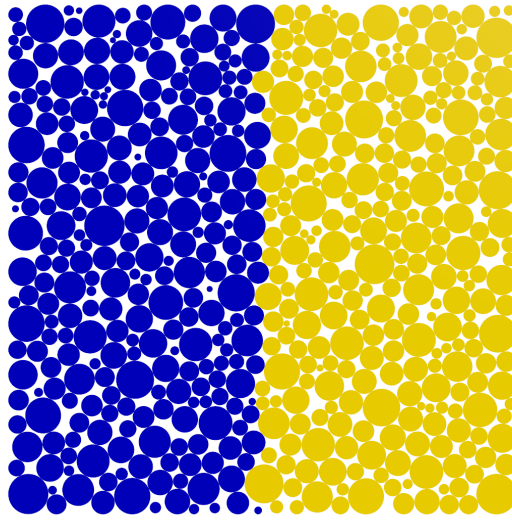


Figure 15: Illustration of how the ice floes are injected: in blue the first ice array (at $t = t_1$), in yellow the second ice array (at $t = t_1 + \Delta t_{\text{inject}}$)

3.6 Sensitivity Test on DEM parameters

See Section 2.2.3 for details on these parameters.

Several sensitivity tests were performed to choose the appropriate values for the DEM time scale and the spring stiffness constant K .

The aim is to identify parameter values that limit the maximum inter-particle overlap to approximately 1mm while maintaining an acceptable computational cost.

3.6.1 Setup of the Sensitivity Test

The setup is simple: a domain with no gravity and no water is created, with a wall. A DEM particle is launched toward the wall at a fixed velocity, representing the ice floe with the maximum diameter used in the experiments. The maximum overlap at the wall, the maximum DEM force, the DEM force impulse, and the CPU time are then recorded.

Simcenter STAR-CCM+

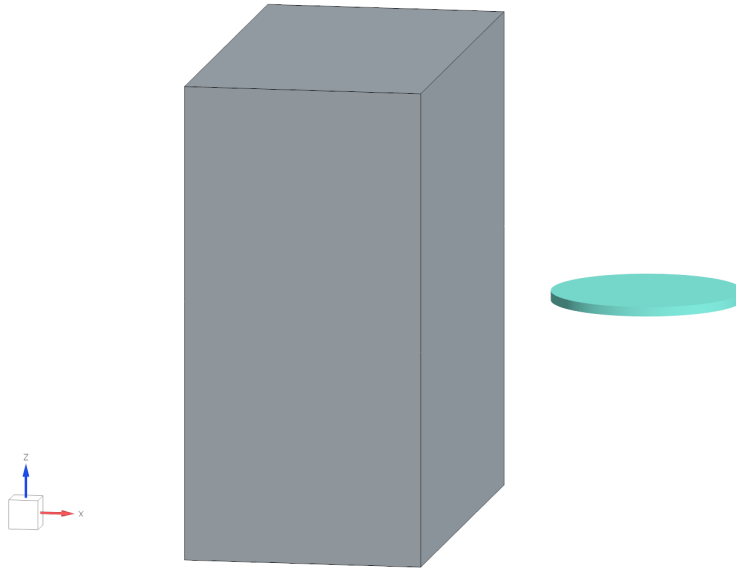


Figure 16: Setup for the sensitivity test.

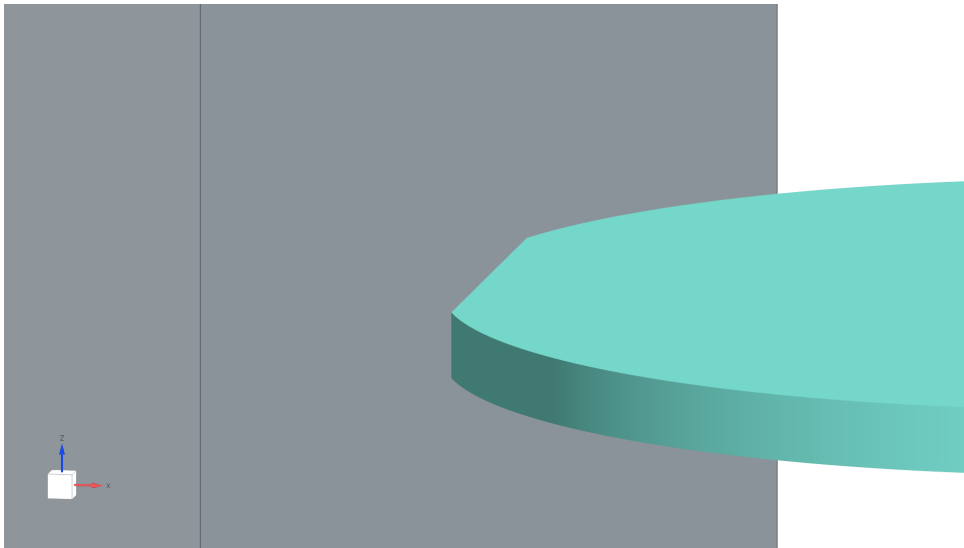


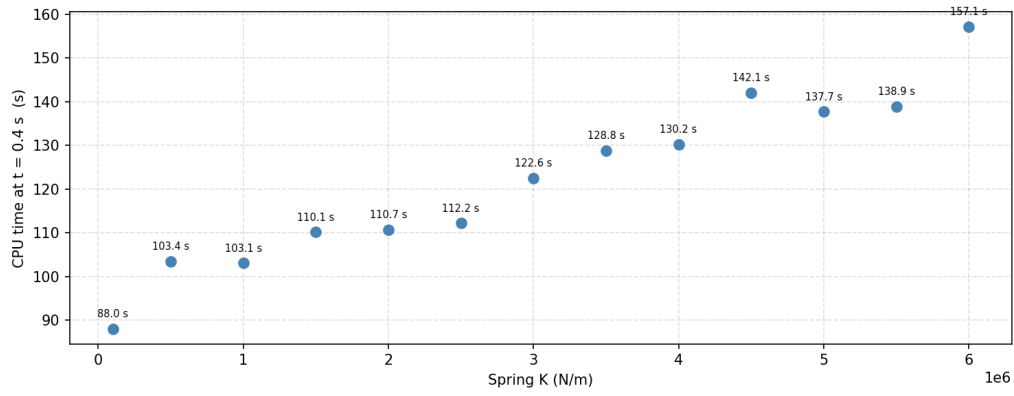
Figure 17: Example of overlap between the wall and the ice floe.

3.6.2 Spring Stiffness Constant K

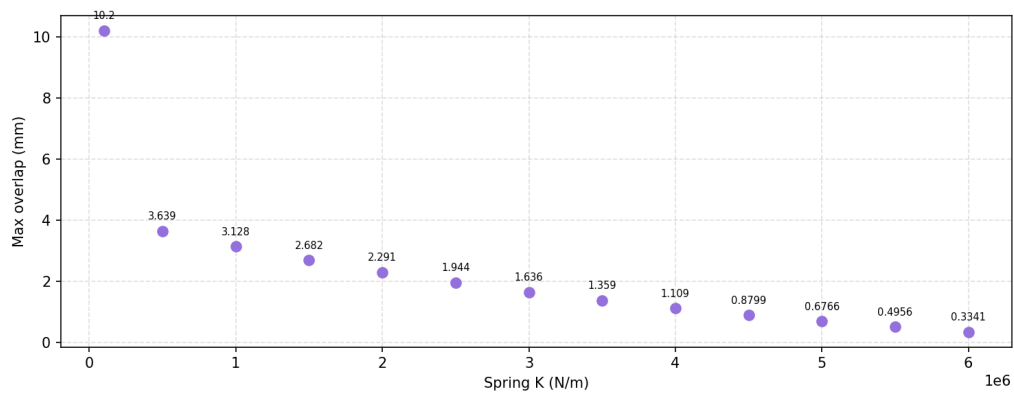
For this test, the time scale was set to 0.4 and the CFD timestep to $dt = 0.01$ s. The spring stiffness K was then tested from 0.1 to 6 MN/m, and each simulation was run for 0.4 s.

The selection of K involves a trade-off between two competing requirements. A high K gives more accurate results, especially for the DEM force impulse, and reduces the overlap. But a high K also makes the DEM time step smaller (see Equation (23)), which significantly increases CPU time.

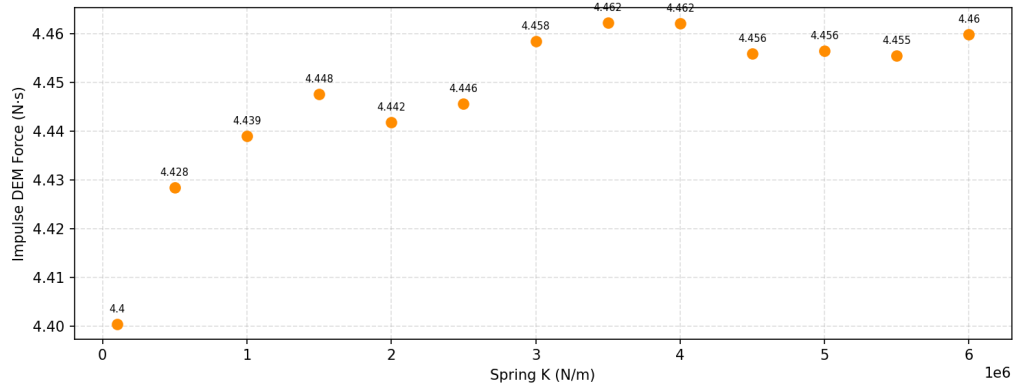
The value $K = 4$ MN/m was chosen because it gives good results for the DEM force impulse, a small enough overlap, and a reasonable CPU time.



(a) CPU time.



(b) Maximum overlap (mm).



(c) DEM force impulse (N.s).

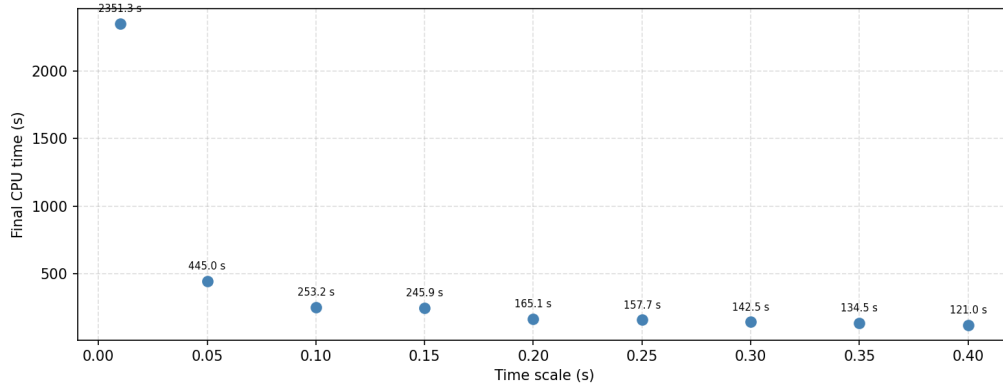
Figure 18: Sensitivity test results for the spring stiffness constant K .

3.6.3 Time Scale

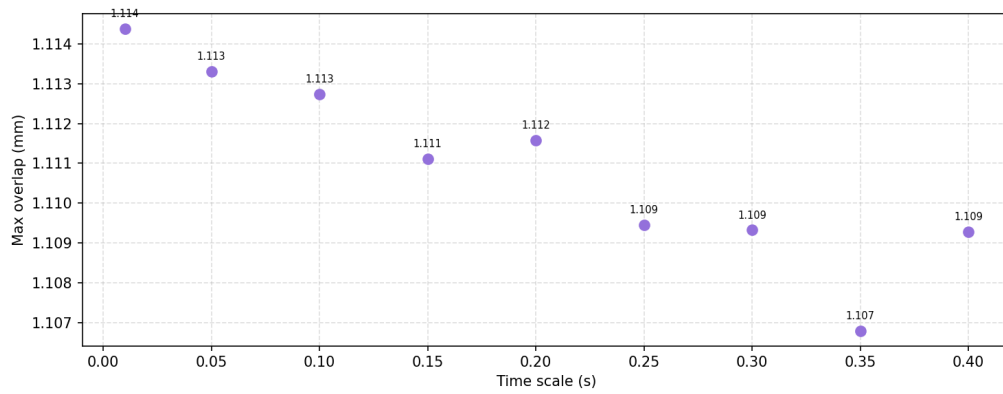
The effect of the time scale t_{scale} was studied by looking at the DEM force impulse, the maximum overlap, and the CPU time. The time scale is used to set the DEM time step in the solver. STAR-CCM+ recommends values between 0.1 and 0.4, but some documentation suggests using values as low as 0.05 or 0.01 [22]. The range from 0.01 to 0.4 was therefore tested, with $dt = 0.01$ s and $K = 4$ MN/m.

The results show that the DEM force impulse and the maximum overlap are almost not

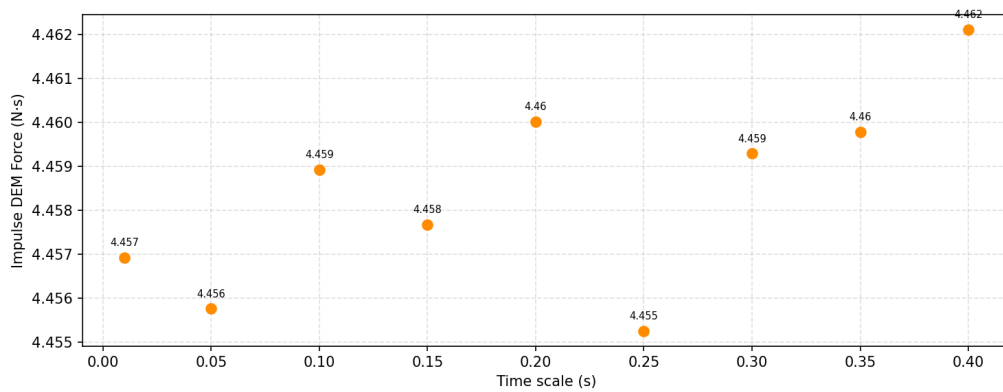
affected by the time scale in this study, with changes below 0.6%. However, the CPU time largely increases when the time scale is reduced. For this reason, $t_{\text{scale}} = 0.4$ was chosen to keep the CPU time as low as possible. a value greater than 0.4 was not chosen in order to remain within the recommendations of STAR-CCM+.



(a) CPU time.



(b) Maximum overlap (mm).



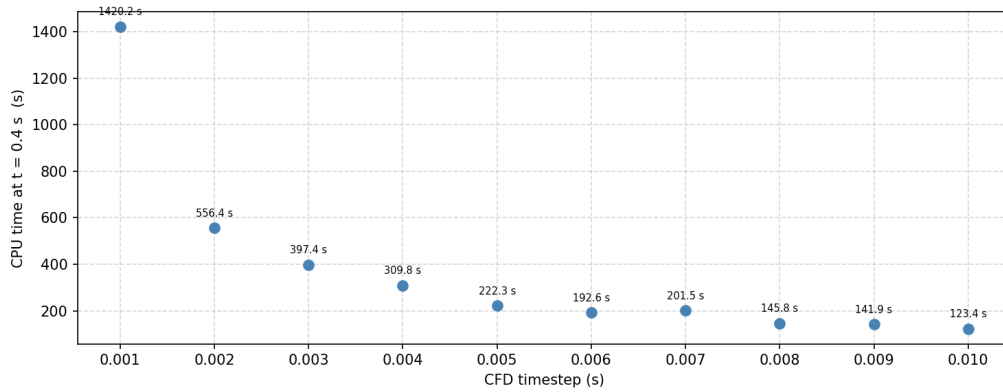
(c) DEM force impulse (N·s).

Figure 19: Sensitivity test results for the time scale t_{scale} .

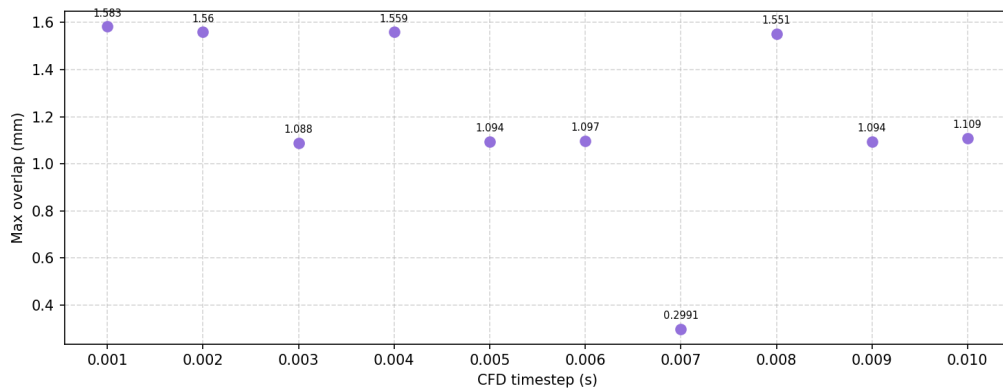
3.6.4 CFD Time Step

A last sensitivity test was done on the CFD time step, with $K = 4 \text{ MN/m}$ and $t_{\text{scale}} = 0.4$. Values from 0.001 s to 0.01 s were tested.

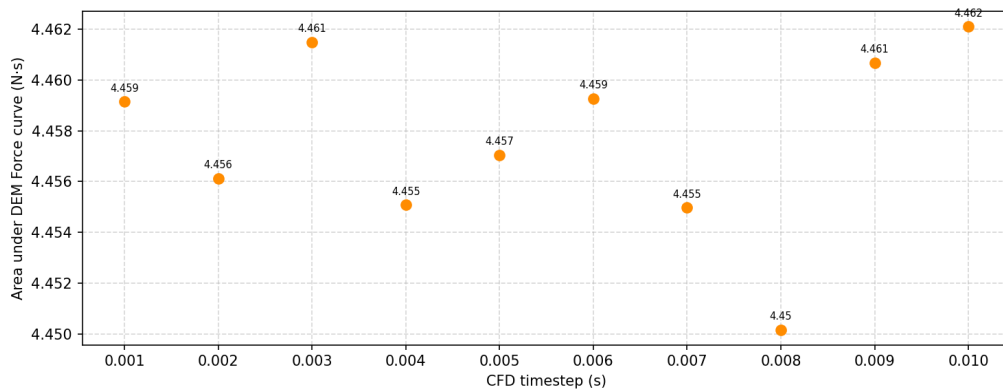
The results show that the CFD time step has very little effect on the DEM force impulse or the maximum overlap, with differences below 1%. Reducing the CFD time step increases the CPU time. Since $dt = 0.01 \text{ s}$ is the time step used in the deep-water CFD study, this value was kept.



(a) CPU time.



(b) Maximum overlap (mm).



(c) DEM force impulse (N·s).

Figure 20: Sensitivity test results for the CFD time step dt .

3.6.5 Conclusion

With a spring stiffness $K = 4 \text{ MN/m}$, a time scale $t_{\text{scale}} = 0.4$, and a CFD time step $dt = 0.01 \text{ s}$, the maximum overlap is close to 1 mm , and the DEM force impulse reaches $4.462 \text{ N}\cdot\text{s}$, compared to a theoretical value of $4.5 \text{ N}\cdot\text{s}$, which represents a relative error of 0.89% . Furthermore, these parameter values significantly reduce the computational time compared to recommended values from literature [22].

4 Results

Simulations were carried out to compare the numerical results with the experimental data reported in [7]. Five different ship speeds were considered, namely $V = 0.4, 0.6, 0.8, 1.0$, and 1.2 m/s, corresponding to Froude numbers of $F_r = 0.06, 0.09, 0.12, 0.15$, and 0.18 , respectively. For each case, an ice floe field was injected over a duration of 40 s. Such a simulation time is required because the contacts between the hull and the ice floes generate force peaks. Therefore, the resistance must be averaged over a sufficiently long time interval in order to obtain representative mean values.

4.1 Added Ice Resistance

4.1.1 Results and comparison with experimental results

The resistance prediction results from the simulations are compared with the experimental results from Guo et al. [7].

The simulation results take the form of spikes due to the contact of the ice floes on the ship hull, as shown in Figure 21(a).

To better visualise the DEM forces acting on the hull, it is more convenient to work with the DEM impulse (unit: N·s) rather than the raw force signal. This approach serves two purposes: it allows one to check whether the simulation has run long enough to reach convergence, and it gives a reliable estimate of the mean DEM force on the ship.

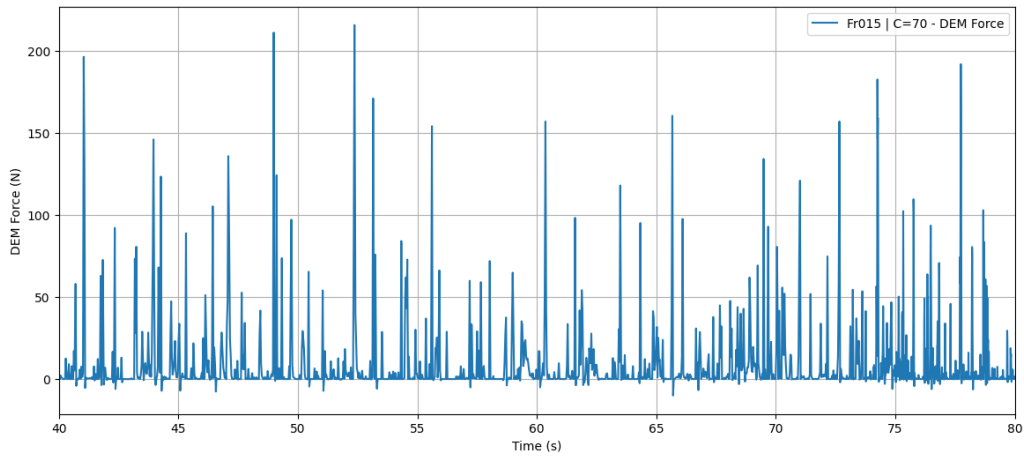
The cumulative DEM impulse is the area under the DEM force curve over time, as shown in Figure 21(b), defined as:

$$I(t) = \int_0^t F_{\text{DEM}}(\tau) d\tau \quad (49)$$

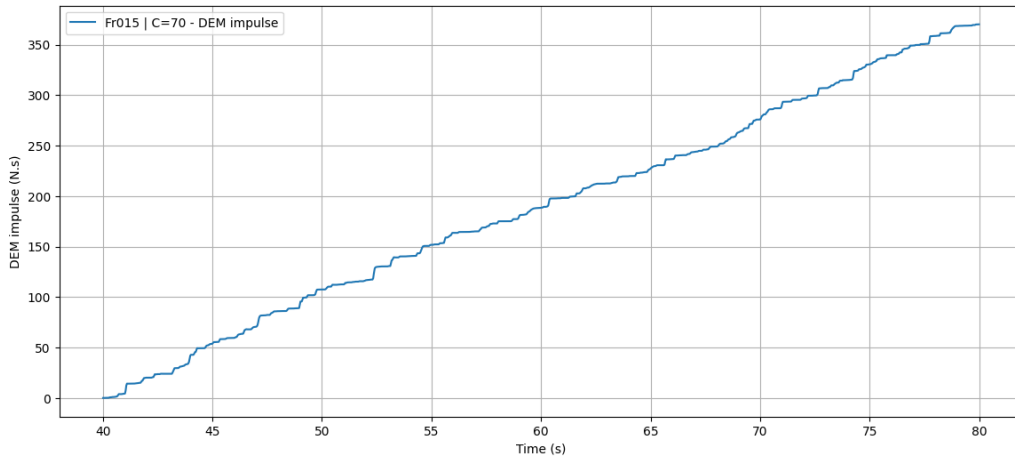
The cumulative ice resistance is then obtained by dividing the cumulative impulse by the elapsed time:

$$R_{\text{ice}}(t) = \frac{I(t)}{t} \quad (50)$$

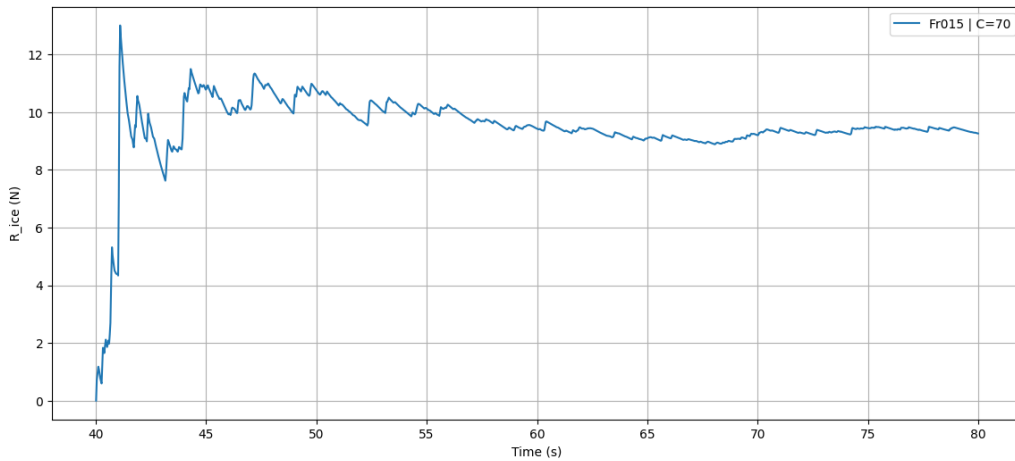
When this quantity stabilises and converges to a constant value, it confirms that the simulation has run long enough and that the mean ice resistance can be read directly from the plateau. As shown in Figure 21c, convergence is reached after approximately 40 seconds, which validates the chosen simulation duration.



(a) Raw DEM force on the hull over time. The spikes correspond to individual ice floe contacts.



(b) Cumulative DEM impulse $I(t)$, representing the time-integrated DEM force on the hull.

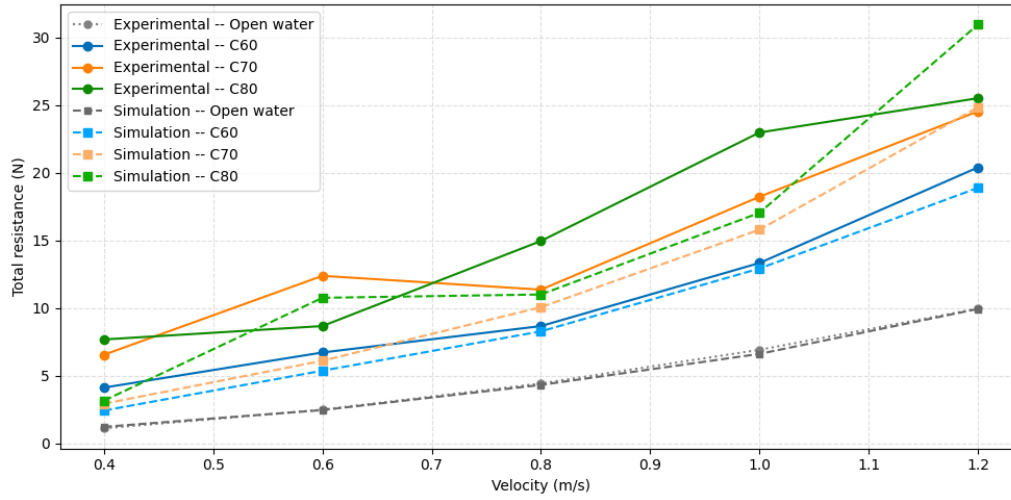


(c) Cumulative ice resistance $R_{ice}(t) = I(t)/t$. Convergence to a constant value confirms that the simulation duration is sufficient.

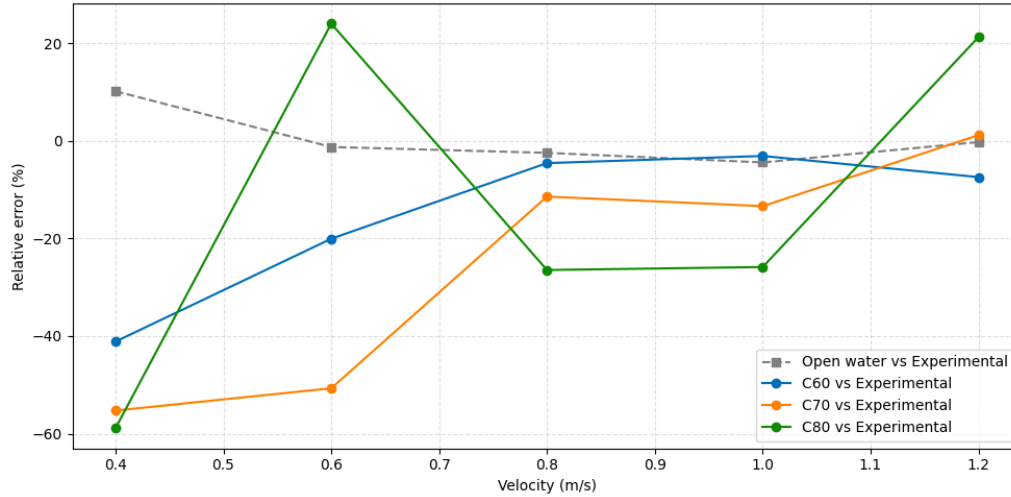
Figure 21: DEM force on the ship hull (a), cumulative DEM impulse (b), and convergence of the mean ice resistance (c).

Simulations were run for all velocities at ice concentrations of 60%, 70% and 80%. The total

resistance values are compared with the experimental results in Figure 22 and Figure 23.



(a) Experimental and simulated total resistance for $c_{ice} = 60\%$, 70% and 80% .

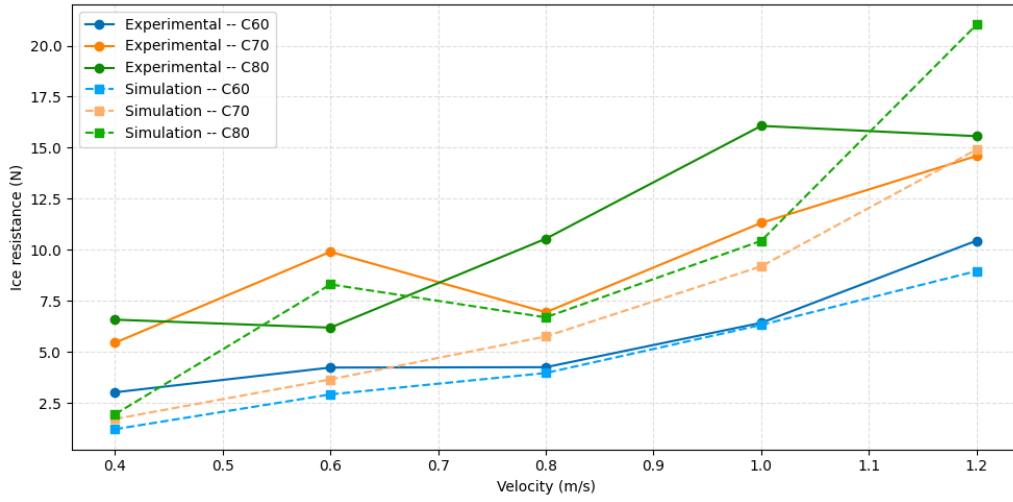


(b) Relative error $\frac{R_{sim}-R_{exp}}{R_{exp}} \times 100$ between experimental and simulated total resistance.

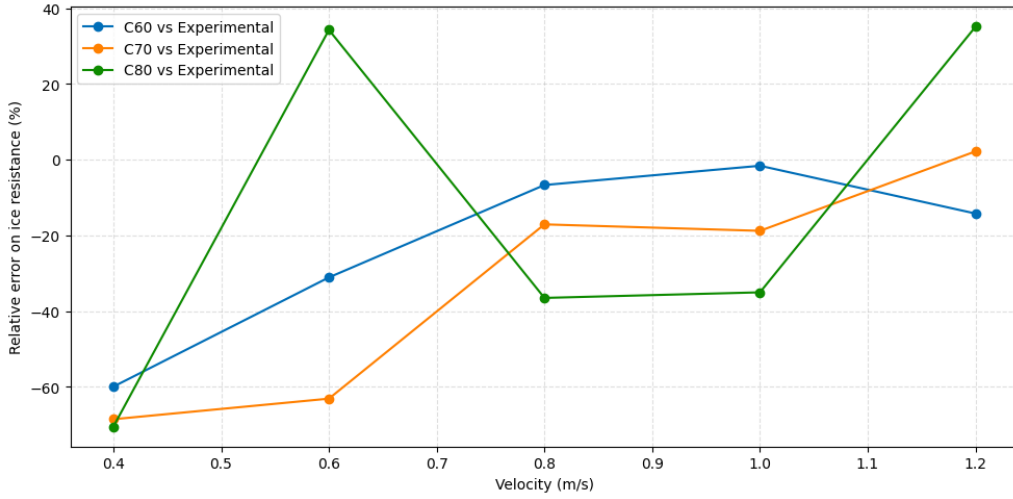
Figure 22: Comparison of total resistance between simulation and experiment (a), and corresponding relative error (b).

Velocity	Open Water	$c_{ice} = 60$	$c_{ice} = 70$	$c_{ice} = 80$
0.4	+10.23%	-41.13%	-55.28%	-58.89%
0.6	-1.21%	-20.03%	-50.71%	+24.08%
0.8	-2.42%	-4.52%	-11.39%	-26.45%
1.0	-4.28%	-4.55%	-14.44%	-26.70%
1.2	-0.21%	-7.39%	+1.21%	+21.33%

Table 7: Relative error between experimental and simulated total resistance $\frac{R_{sim}-R_{exp}}{R_{exp}} \times 100$



(a) Experimental and simulated Ice resistance for $c_{ice} = 60\%$, 70% and 80% .



(b) Relative error $\frac{R_{ice,sim} - R_{ice,exp}}{R_{ice,exp}} \times 100$ between experimental and simulated ice resistance.

Figure 23: Comparison of ice resistance between simulation and experiment (a), and corresponding relative error (b).

First, similar behaviors are observed for $c_{ice} = 60\%$ and 70% : the simulations tend to underestimate the resistance (both the total resistance and the ice resistance alone). At higher velocities (above 0.8 m/s), the agreement with the experimental data is relatively good, with relative errors below 15% on the total resistance. However, at lower velocities (0.4 and 0.6 m/s), the relative error increases significantly, reaching values between 20% and 60% . A possible explanation for these large inconsistencies at low speed is discussed in Section 4.2.2.

At higher ice concentration ($c_{ice} = 80\%$), the simulations tend to underestimate for 0.8 and 1.0 m/s but overestimate for 1.2 m/s, with relative errors approaching 25% on the total resistance. Hypotheses about these errors are discussed in Section 4.2.3.

Another noteworthy result is the resonance-like behavior observed in the simulation at $V = 0.6$ m/s for an ice concentration of $c_{ice} = 80\%$. A similar phenomenon was documented in the experimental campaign of Guo et al. [7], also at $V = 0.6$ m/s, but for a lower concentration of $c_{ice} = 70\%$. At this specific velocity, instead of pushing the ice floes aside, the ship model carries them along with it, causing an abrupt increase in total resistance:

“In the special region around 0.6 m/s, the frequency of the ship model encountering the pack ice occasionally shows a certain equilibrium between a few factors, namely the friction between the body and the ice, and the water resistance on the ice. Hence, instead of pushing the ice aside, the ship model brings the ice into motion along with it, resulting in an abrupt increase in the total resistance as compared with the total resistance at 0.4 m/s. The total resistance reaches its maximum around 0.6 m/s.” [7]

The simulation reproduces this mechanism at the same critical velocity, lending credibility to the numerical approach. While the resonance is triggered at $c_{ice} = 70\%$ in the experiment and at $c_{ice} = 80\%$ in the simulation, both cases exhibit the abrupt resistance peak at $V = 0.6$ m/s, confirming that the simulation captures this specific ice–ship interaction behavior.

4.2 Ice Floe Behavior upon Contact with the Ship

4.2.1 Rotating, Submerging and Sliding

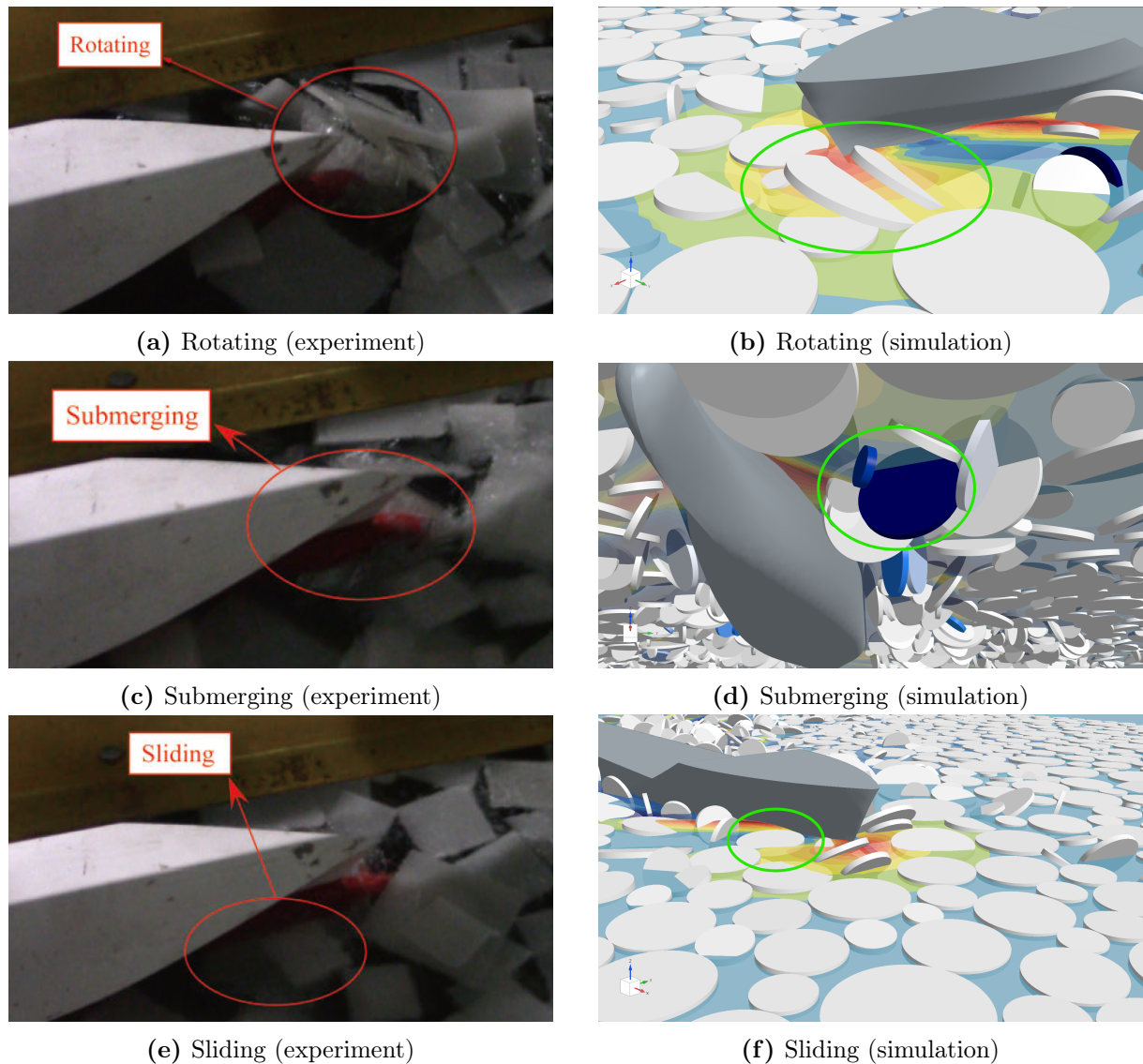


Figure 24: Rotating, submerging and sliding of ice floes: experimental results [7] (left) and simulation in STAR-CCM+ (right).

Different behaviors can be observed when the ice floes come into contact with the ship. From experimental results [7], three main behaviors can be identified: rotating, submerging, and sliding, that can be also observed in the simulations.

4.2.2 Non-Realistic Behavior at Low Velocity

As explained in Section 2.2.5, the ice floes are represented as DEM particles, a simplification that neglects several of the hydrodynamic forces exerted by the fluid. Scaling the moment of inertia and adding a drag torque artificially restores some of these missing effects, but others remain absent, in particular the hydrostatic restoring moment that tends to keep a floating floe horizontal.

This limitation has a direct consequence on the simulated ship–ice interaction. Guo et al. [7] observed that at low speed (around 0.4 m/s) the floes are only slowly pushed aside by the hull, whereas at higher speed (around 1.0 m/s) they are pushed aside more abruptly and tend to rotate and submerge. At low speed, in other words, the floes remain essentially horizontal, with the axis of the cylindrical particles staying vertical. In the present simulations, however, the absence of the restoring moment means that the slightest contact with the hull is enough to make a floe rotate. This premature rotation interrupts the chain reaction through which the floes are normally pushed against one another: the upstream floes no longer transmit their thrust to the floes in contact with the hull, which leads to an underestimation of the ice resistance.

Overall, the ice floes should mostly slide along the hull (Figure 24f), whereas in the current simulations they rotate (Figure 24b) and submerge (Figure 24d) more frequently than observed experimentally. One way to improve the model would be to change the particle model. See Section 5.

4.2.3 Non-Realistic Behavior at High Concentration

At 80% ice concentration, significant errors are observed in the ice resistance predictions. Three possible hypotheses are discussed below.

First, the shape of the ice floes may play a role. Cylindrical floes were selected on the grounds that shape should not significantly affect the results [16]; however, this assumption was validated only up to a concentration of 70%, not 80%. Circular ice floes allow lower maximum surface coverage than square ones for the same size distribution, since circles do not tile the plane perfectly. This geometric limitation becomes critical at 80% concentration: the floes exhibit pronounced rebound behavior, which manifests either as sharp spikes in the DEM impulse — and thus a strong overestimation of resistance — or as a large number of floes being pushed laterally away from the ship, reducing contact and potentially underestimating the resistance.

Second, the absence of an ice channel in the simulation may affect the rebound dynamics between ice floes. In the experimental setup, an ice channel confines the domain laterally, meaning that floes pushed sideways by the ship are forced back by the channel walls. In the simulation, however, no such boundary condition exists. As a result, floes near the lateral boundaries ($y = \pm y_{max}$) are displaced outward through floe–floe interactions without any restoring effect, which likely leads to an overestimation of the resistance at high concentration (see Figure 25b).

Third, initial overlap between ice floes is observed in the experimental ice floe field, where some floes are stacked on top of others, as visible in Figure 25. This may alter the overall behavior of the ice field, and in particular the effective ice concentration seen by the ship.

These hypotheses could be investigated in future work to either confirm or rule out their respective contributions to the observed discrepancies at high ice concentration.

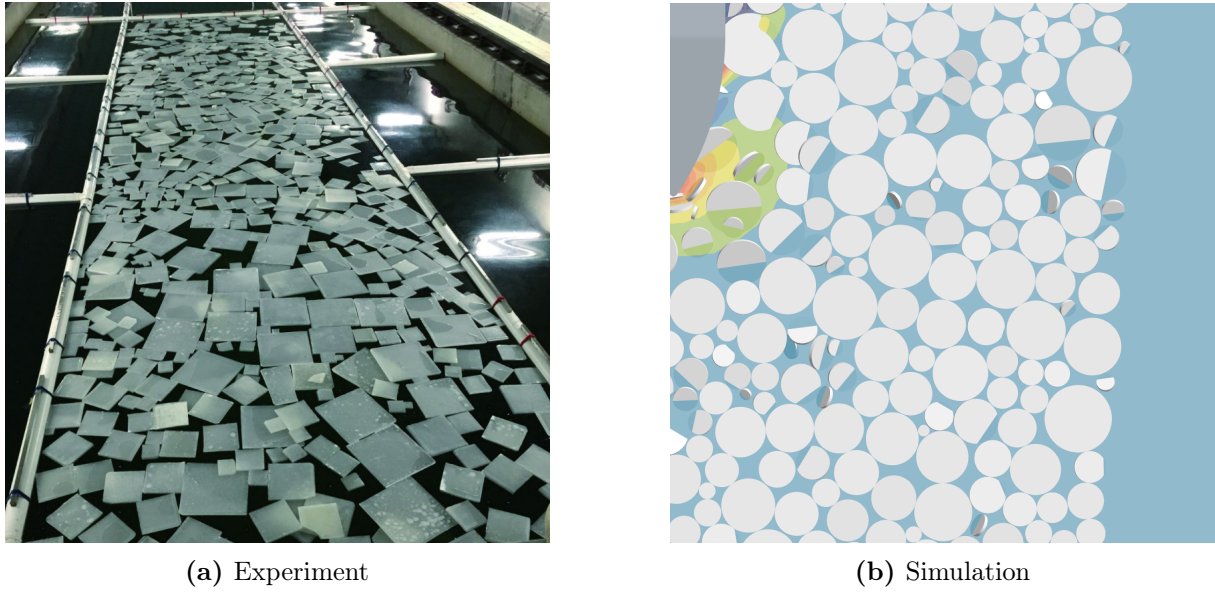


Figure 25: Ice floe fields

4.3 Impact on the ship

The regions of impact can be visualised in STAR-CCM+. In Figure 26, the root-mean-square (RMS) of the DEM contact force is plotted over the hull surface. The RMS provides a representative measure of the sustained loading at each location, giving more weight to the strongest impacts. It is defined as

$$F_{\text{RMS}} = \sqrt{\frac{1}{T_{\text{avg}}} \int_0^{T_{\text{avg}}} F_{\text{DEM}}^2(t) dt}, \quad (51)$$

where $F_{\text{DEM}}(t)$ is the DEM force acting on a given region of the hull and T is the averaging period.

The main impact zone is located at the edge of the bow, i.e. the most forward part of the hull at the free surface, where the ship first meets the oncoming floes. The bulbous bow is also strongly loaded. Finally, the sides of the hull experience some impacts as well, even near the keel; these originate from ice floes that have been submerged and pass beneath the hull before resurfacing.

Overall, the principal impacts occur on the frontal area of the hull. Beyond the estimation of the added resistance, the simulation therefore also allows the additional structural loads acting on the hull to be estimated. This is essential information for the design of a vessel intended to operate in Arctic waters, where these ice-induced loads can drive premature wear or structural damage. (see Figure 2).

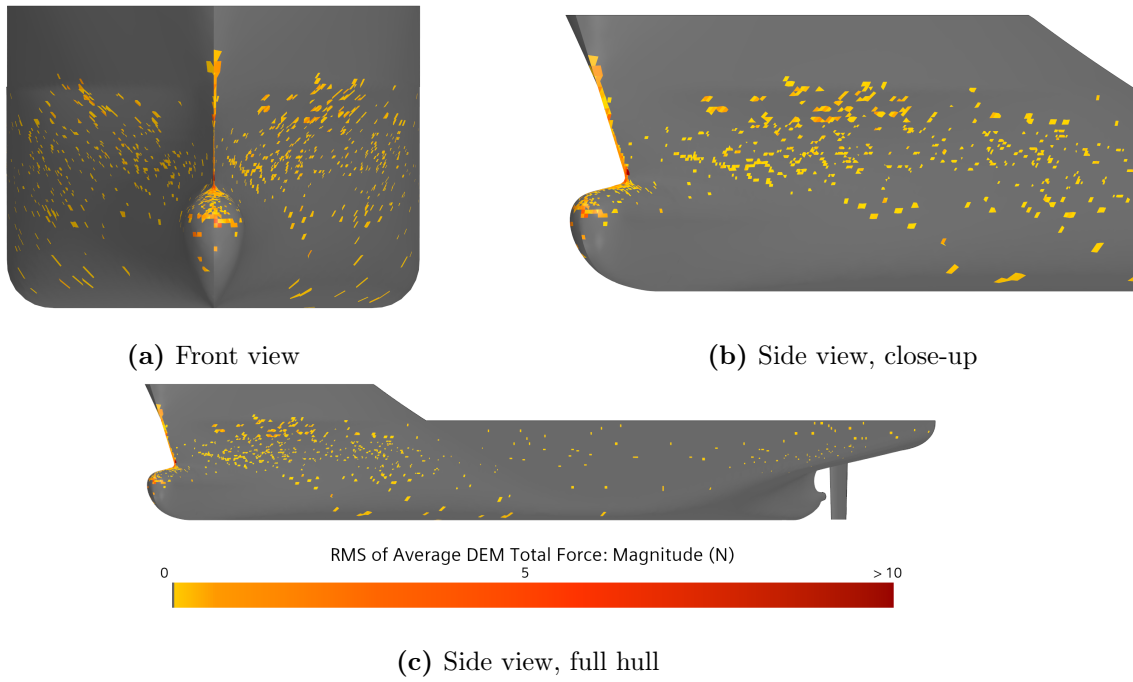


Figure 26: Distribution of the ice-floe impacts on the hull.

5 Additional work: particle clumps model investigation.

A preliminary study was conducted on the particle clumps model in STAR-CCM+. The objective of this study is to explore the capabilities offered by this model, and the issues it could address in the study of ships navigating in icy conditions.

The particle clumps model allows multiple sub-DEM particles in the form of spheres to represent a single ice floe. As a result, for the fluid-particle interaction, each sub-particle has its own interaction with the fluid.

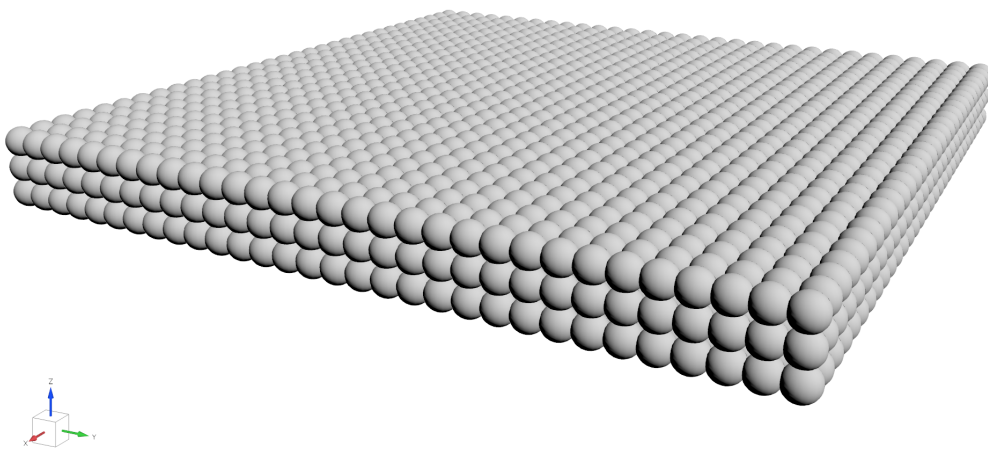


Figure 27: Example of an ice floe using the clumps model, size = $0.2 \times 0.2 \times 0.02$ m

This model therefore allows a more realistic representation of the fluid-particle interaction, as the surface forces are estimated from the individual sub-DEM particles. Consequently, if an ice floe is injected with an angle (about the x or y axis), it will return to its equilibrium position.

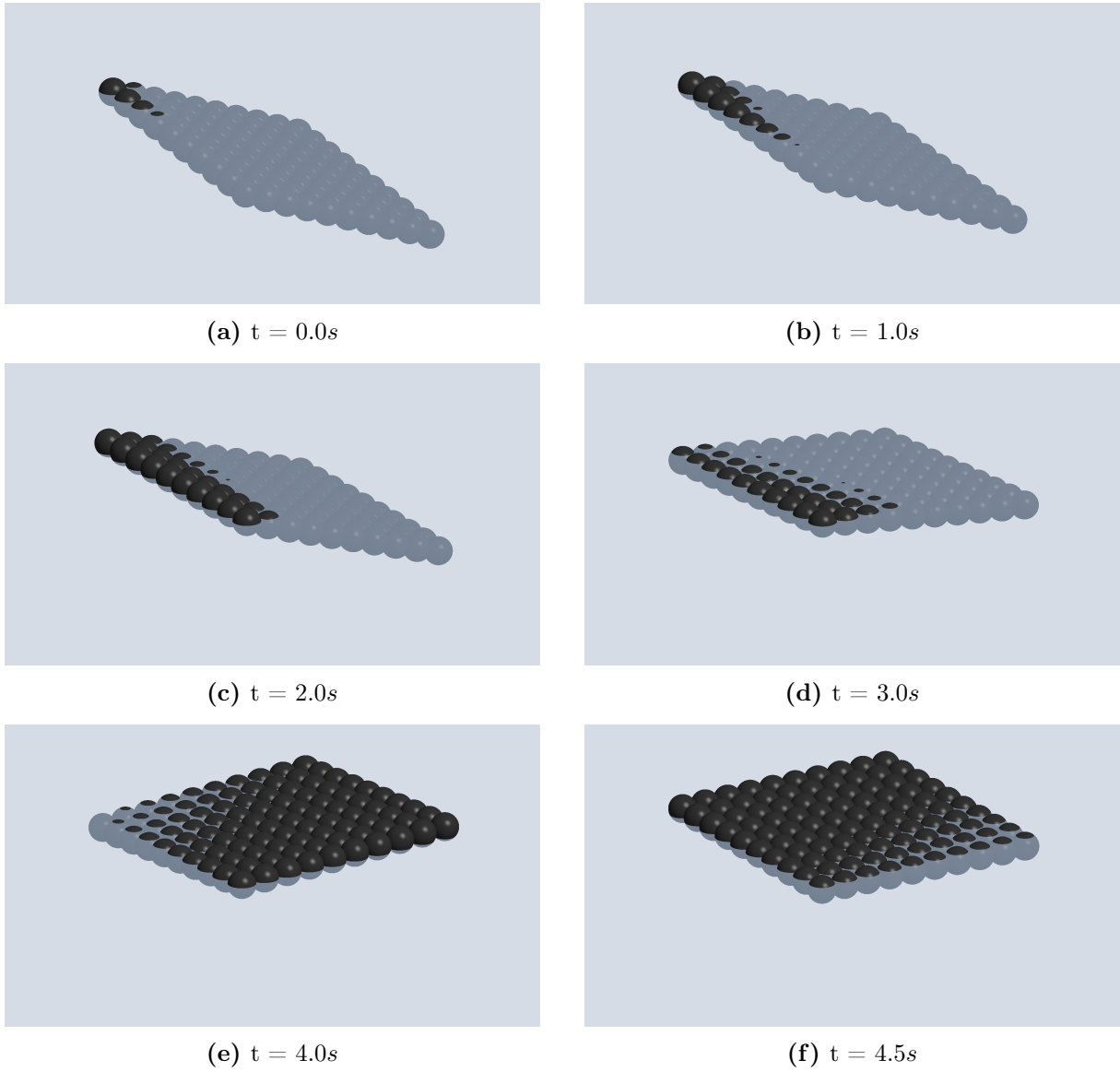


Figure 28: Evolution of the ice floe position over time to the equilibrium, using particle clumps model

6 Conclusion

A CFD–DEM model was developed in STAR-CCM+ to predict the added resistance of a ship navigating through floating ice floes and validated on the KCS hull against the experimental results for ice concentrations of 60 %, 70 % and 80 % at five velocities ($F_r = 0.06\text{--}0.18$).

For 60% and 70%, the model tends to underestimate the total resistance. Numerical results are closer to experimental results at higher speeds (above 0.8 m/s), with relative errors near 10% or below, but they degrade at low speed (0.4 and 0.6 m/s), where relative errors on the total resistance reach 20 to 60%. This is attributed to the simplified hydrodynamic representation of the floes: the hydrostatic restoring moment that keeps a floe horizontal is absent, so the slightest contact makes a floe rotate prematurely. This interrupts the chain reaction through which upstream floes transmit their thrust to those in contact with the hull, leading to the underestimation. The floes should mostly slide along the hull, whereas in the simulations they rotate and submerge too often.

For $c_{ice} = 80\%$, the errors between simulated and experimental resistance are larger than at lower concentrations. There is still the floe-rotation problem identified at 60% and 70%. In addition, two supplementary factors could contribute as well: first, the shape of the ice floes could prove critical at high concentrations: circular floes allow lower maximum surface coverage than square ones for the same size distribution, which affects the contact behavior (for example, more inter-floe contacts for circular than square ones). At 80%, the circular floes are nearly in permanent contact; the resistance imparted by the ice floes on the ship no longer acts individually, but rather as a chain reaction with several floes contributing simultaneously. Secondly, the absence of an ice channel could be critical for comparison with the experimental data.

Despite these discrepancies, a resonance-like behavior consistent with experimental observations is nonetheless captured at $V = 0.6$ m/s, as discussed above.

Additionally, the moment of inertia scaling was modified to account for the added mass moment, but the value of this scaling depends strongly on the diameter of the ice floes. However, it was not possible to implement a per-floe moment of inertia scaling, so an average value was used instead.

Therefore, the current model allows one to evaluate, with relative reliability, the resistance to motion of a ship navigating in ice floes at high speeds and for ice concentrations of 60 % and 70 %, but shows its limitations when it comes to lower velocities and high concentrations.

7 Future Work

The prediction of ship resistance in ice remains an open research area with several promising directions.

First, regarding the limitations identified in this study, the choice of particle model is a likely source of error. Each ice floe is currently represented as a cylinder modelled by a single DEM particle, which prevents any hydrodynamic interaction with the surrounding fluid at the floe scale. One improvement would be to adopt a particle-clumps model, where each floe is composed of several sub-elements, enabling a more faithful representation of floe hydrodynamics. An initial study has been made (see Appendix 5)

Furthermore, cylindrical floe shapes were used on the basis that shape has been reported to have little influence on resistance [16]; however, the present results suggest that this conclusion may not hold at high ice concentrations. It could therefore be relevant to investigate square floes and, more importantly, realistic broken-ice shapes (random polygons), which would also allow concentrations approaching 100% to be simulated. A similar study has already been conducted using FEM [10], and could be extended to STAR-CCM+ using DEM with particle clumps, for which a preliminary study has been carried out with promising results.

A further avenue would be to investigate the CFD–DEM coupling scheme in greater depth. Several works consider *one-way* coupling sufficient for resistance prediction [16, 17], and direct comparisons show no notable difference in mean resistance between *one-way* and *two-way* schemes [17]. However, this conclusion has only been established at moderate concentrations: Huang et al. [16] justify the use of *one-way* coupling on the grounds that the concentration is not high enough to produce significant wave reflection, and limit their study to a maximum of 70%. At very high concentrations, where floes are densely packed and force chains can form between them, the feedback of the ice motion on the flow field has not been evaluated. It would therefore be worthwhile to assess whether *two-way* coupling becomes necessary in this regime.

Moreover, studying a ship in self-propulsion within ice floes could prove interesting. Indeed, in real-world scenarios, vessels cannot maintain a constant speed within an ice-floe field. It would also be interesting to conduct manoeuvrability tests.

Finally, the study of resistance in a brash-ice channel represents another interesting direction. The aspect ratio of brash-ice fragments is much closer to unity, meaning that single-particle DEM would already provide a more realistic representation without requiring particle clumps. Beyond the larger number of particles involved, the principal interest of this configuration lies in the boundary effects introduced by the ice channel walls.

References

- [1] AMAP. *Arctic Climate Issues 2011: Changes in Arctic Snow, Water, Ice and Permafrost*. Tech. rep. Oslo, Norway: Arctic Monitoring and Assessment Programme (AMAP), 2011. URL: <https://www.amap.no/documents/doc/arctic-climate-issues-2011-changes-in-arctic-snow-water-ice-and-permafrost/129>.
- [2] Katherine Si. *China Launches First Arctic Container Service to Europe*. <https://www.seatrade-maritime.com/containers/china-launches-first-arctic-container-service-to-europe>. Accessed: 2026-06-03. Seatrade Maritime News, 2025.
- [3] YES Containers. *Arctic Express Shipping Route*. <https://yescontainers.com/arctic-express-shipping-route/>. Accessed: 2026-06-03. 2025.
- [4] Nial McCarthy. *Global Warming Opens Arctic Passage For Container Ships [Infographic]*. Forbes. Accessed: 2026-06-11. 2018. URL: <https://www.forbes.com/sites/niallmccarthy/2018/08/27/global-warming-opens-arctic-passage-for-container-ships-infographic/>.
- [5] Meridien. *Bulbous Bow Repair*. <https://www.meridien.cc/en/division/maritime-division/portfolio/9-ship-repair/19-afloat-repairs/55-bulbous-bow-repair>. Accessed: 2026-06-02. 2026.
- [6] Rodrigue de Schaetzen et al. *AUTO-IceNav: A Local Navigation Strategy for Autonomous Surface Ships in Broken Ice Fields*. arXiv preprint arXiv:2411.17155. 2024. arXiv: [2411.17155](https://arxiv.org/abs/2411.17155). URL: <https://arxiv.org/abs/2411.17155>.
- [7] Chun-yu Guo et al. “Experimental Investigation of the Resistance Performance and Heave and Pitch Motions of Ice-Going Container Ship under Pack Ice Conditions”. In: *China Ocean Engineering* 32.2 (2018), pp. 169–178. DOI: [10.1007/s13344-018-0018-9](https://doi.org/10.1007/s13344-018-0018-9).
- [8] Wan-Zhen Luo et al. “Experimental Research on Resistance and Motion Attitude Variation of Ship-Wave-Ice Interaction in Marginal Ice Zones”. In: *Marine Structures* 58 (2018), pp. 399–415. DOI: [10.1016/j.marstruc.2017.12.013](https://doi.org/10.1016/j.marstruc.2017.12.013).
- [9] Moon-Chan Kim, Won-Joon Lee, and Yong-Jin Shin. “Comparative Study on the Resistance Performance of an Icebreaking Cargo Vessel according to the Variation of Waterline Angles in Pack Ice Conditions”. In: *International Journal of Naval Architecture and Ocean Engineering* 6.4 (2014), pp. 876–893. DOI: [10.2478/IJNAOE-2013-0219](https://doi.org/10.2478/IJNAOE-2013-0219).
- [10] Jeong-Hwan Kim et al. “Numerical Simulation of Ice Impacts on Ship Hulls in Broken Ice Fields”. In: *Ocean Engineering* 180 (2019), pp. 162–174. DOI: [10.1016/j.oceaneng.2019.03.043](https://doi.org/10.1016/j.oceaneng.2019.03.043).
- [11] Tom Yulsman. *How to Hitch a Ride on an Arctic Ice Floe*. Ocean Deeply. Accessed: 2026-06-11. 2017. URL: <https://deeply.thenewhumanitarian.org/oceans/articles/2017/06/12/how-to-hitch-a-ride-on-an-arctic-ice-floe>.
- [12] Moon-Chan Kim et al. “Numerical and Experimental Investigation of the Resistance Performance of an Icebreaking Cargo Vessel in Pack Ice Conditions”. In: *International Journal of Naval Architecture and Ocean Engineering* 5.1 (2013), pp. 116–131. DOI: [10.2478/IJNAOE-2013-0121](https://doi.org/10.2478/IJNAOE-2013-0121).
- [13] Chun-yu Guo et al. “Numerical Simulation on the Resistance Performance of Ice-Going Container Ship Under Brash Ice Conditions”. In: *China Ocean Engineering* 32.5 (2018), pp. 546–556. DOI: [10.1007/s13344-018-0057-2](https://doi.org/10.1007/s13344-018-0057-2).

- [14] Erik Vroegrijk. “Validation of CFD+DEM against Measured Data”. In: *Proceedings of the ASME 2015 34th International Conference on Ocean, Offshore and Arctic Engineering (OMAE2015)*. OMAE2015-41770, V008T07A021. 2015. DOI: [10.1115/OMAE2015-41770](https://doi.org/10.1115/OMAE2015-41770).
- [15] Luofeng Huang and Giles Thomas. “Simulation of Wave Interaction with a Circular Ice Floe”. In: *Journal of Offshore Mechanics and Arctic Engineering* 141.4 (2019), p. 041302. DOI: [10.1115/1.4042096](https://doi.org/10.1115/1.4042096).
- [16] Luofeng Huang et al. “Ship Resistance when Operating in Floating Ice Floes: A Combined CFD&DEM Approach”. In: *Marine Structures* 74 (2020), p. 102817. DOI: [10.1016/j.marstruc.2020.102817](https://doi.org/10.1016/j.marstruc.2020.102817).
- [17] Wanzhen Luo et al. “Numerical Simulation of an Ice-Strengthened Bulk Carrier in Brash Ice Channel”. In: *Ocean Engineering* 196 (2020), p. 106830. DOI: [10.1016/j.oceaneng.2019.106830](https://doi.org/10.1016/j.oceaneng.2019.106830).
- [18] Jianing Zhang et al. “CFD-DEM Based Full-Scale Ship-Ice Interaction Research under FSICR Ice Condition in Restricted Brash Ice Channel”. In: *Cold Regions Science and Technology* 194 (2022), p. 103454. DOI: [10.1016/j.coldregions.2021.103454](https://doi.org/10.1016/j.coldregions.2021.103454).
- [19] P. Tuovinen. “The Size Distribution of Ice Blocks in a Broken Channel”. MA thesis. Teknillinen korkeakoulu (Helsinki University of Technology), 1979.
- [20] V. Planque et al. “Experimental Measurement and Expression of Atmospheric Ice Young’s Modulus According to Its Density”. In: *Cold Regions Science and Technology* (2023). URL: <https://www.sciencedirect.com/science/article/abs/pii/S0165232X23001209>.
- [21] P. A. Cundall and O. D. L. Strack. “A Discrete Numerical Model for Granular Assemblies”. In: *Géotechnique* 29.1 (1979), pp. 47–65. DOI: [10.1680/geot.1979.29.1.47](https://doi.org/10.1680/geot.1979.29.1.47).
- [22] Mark A. Hopkins and Jukka Tuhkuri. “Compression of Floating Ice Fields”. In: *Journal of Geophysical Research: Oceans* 104.C7 (1999), pp. 15815–15825. DOI: [10.1029/1999JC900127](https://doi.org/10.1029/1999JC900127).
- [23] Anders Damsgaard, Alistair Adcroft, and Olga V. Sergienko. “Application of Discrete-Element Methods to Approximate Sea Ice Dynamics”. In: *Journal of Advances in Modeling Earth Systems* 10.9 (2018), pp. 2228–2244. DOI: [10.1029/2018MS001299](https://doi.org/10.1029/2018MS001299).
- [24] David C. Wilcox. *Turbulence Modeling for CFD*. 3rd ed. La Cañada, CA: DCW Industries, 2006.
- [25] C. W. Hirt and B. D. Nichols. “Volume of Fluid (VOF) Method for the Dynamics of Free Boundaries”. In: *Journal of Computational Physics* 39.1 (1981), pp. 201–225. DOI: [10.1016/0021-9991\(81\)90145-5](https://doi.org/10.1016/0021-9991(81)90145-5).
- [26] *Marine Resistance Prediction: KCS Hull with a Rudder*. STAR-CCM+. n.d.
- [27] Alexandr I. Korotkin. *Added Masses of Ship Structures*. Fluid Mechanics and Its Applications. Springer, 2007. URL: <https://link.springer.com/book/9781402094316>.
- [28] *Modelling and Analysis of Marine Operations*. DNV-RP-H103. Det Norske Veritas (DNV). 2011. URL: <https://rules.dnv.com/docs/pdf/DNVPM/codes/docs/2011-04/RP-H103.pdf>.
- [29] Joel H. Ferziger, Milovan Perić, and Robert L. Street. *Computational Methods for Fluid Dynamics*. 4th ed. Springer International Publishing, 2020. ISBN: 978-3-319-99691-2. DOI: [10.1007/978-3-319-99693-6](https://doi.org/10.1007/978-3-319-99693-6).
- [30] Henrik Rusche. “Computational Fluid Dynamics of Dispersed Two-Phase Flows at High Phase Fractions”. PhD thesis. Imperial College London, 2003. URL: <https://spiral.imperial.ac.uk/handle/10044/1/8110>.

- [31] *Volume of Fluid Method*. STAR-CCM+. n.d.
- [32] *Discrete Element Method (DEM)*. STAR-CCM+. n.d.
- [33] *Contact Force*. STAR-CCM+. n.d.
- [34] ITTC Resistance Committee. *ITTC – Practical Guidelines for Ship Resistance CFD*. Recommended Procedures and Guidelines, Revision 03 7.5-02-02-01. International Towing Tank Conference, 2011. URL: <https://www.ittc.info/media/1217/75-02-02-01.pdf>.
- [35] J. Gordon Leishman. *Introduction to Aerospace Flight Vehicles: Turbulent Flows*. Embry-Riddle Aeronautical University, 2022. URL: <https://eaglepubs.erau.edu/introductiontoaerospaceflightvehicles/chapter/turbulent-flows/>.

A Code

A.1 Ice Floe field random placement algorithm.

See Section 3.5.3. For this first algorithm, ice floes are randomly placed in decreasing order of diameter. It is fast but cannot reach high ice concentration ($c_{ice} = 60\%$ or less).

For example, the ice floe field generated for the KCS (40s at maximum velocity 1.2m/s) took 26 min.

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import numpy as np
4  from matplotlib.patches import Circle, Rectangle
5  from pathlib import Path
6  import math
7  import os
8  import time
9
10
11  # =====
12  # PARAMETERS - edit this section only
13  # =====
14
15  # Physics
16  rho_water = 997.561
17  rho_ice   = 900
18  g         = 9.81
19
20  # Ship
21  Lpp = 4.367
22
23  # Injection geometry
24  x_injection = 7.0    # x-position of the injection plane [m]
25  y_injection = 5.0    # domain half-width in y [m]
26
27  # Velocities to generate injection CSVs for
28  velocities = np.array([0.2, 0.4, 0.6, 0.8, 1.0, 1.2]) # [m/s]
29
30  # Timing
31  time_step      = 0.01 # CFD time step [s]
32  time_first_impact = 40.0 # time when ice first reaches the ship bow [s]
33  duration_impact  = 40.0 # duration of the ice injection phase [s]
34
35  # Floe geometry
36  thickness      = 0.02 # ice thickness [m]
37  ice_concentration = 0.60 # target ice concentration (0-1)
38
39  # Size distribution - truncated log-normal on diameters
40  min_diameter    = 0.05 # [m]
41  max_diameter    = 0.40 # [m]
```

```
42 mu_diameter      = -1.59    # mean of ln(diameter)
43 sigma_diameter   =  0.45    # std  of ln(diameter)
44
45 # Output folder
46 OUTPUT_DIR = Path(__file__).resolve().parent /
47             ↪ rf"output_IF_random_placement/C{int(ice_concentration * 100)}"
48 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
49
50 t_start = time.perf_counter()
51
52 # =====
53 # BUILD ONE LARGE ICE FIELD FOR THE MAXIMUM VELOCITY
54 # =====
55
56 max_velocity = np.max(velocities)
57
58 # The ice field must cover for {duration_impact} s at the maximum velocity
59 x_output = -duration_impact * max_velocity + x_injection
60
61 coords = np.array([
62     [x_output, -y_injection],
63     [x_injection, y_injection]
64 ])
65
66 print("=" * 60)
67 print("Large ice field domain")
68 print("=" * 60)
69 print(f"Velocities:           {velocities}")
70 print(f"Maximum velocity:      {max_velocity:.3f} m/s")
71 print(f"Injection duration:     {duration_impact:.2f} s")
72 print(f"x_output:              {x_output:.3f} m")
73 print(f"coords =                {coords}")
74 print()
75
76
77 # =====
78 # GENERATE ICE FLOES
79 # =====
80
81 area_domain = (coords[1][0] - coords[0][0]) * (coords[1][1] - coords[0][1])
82
83 ice_floes = []
84
85
86 def ice_floes_diameter_distribution(
87     ice_floes,
88     ice_concentration,
89     area_domain,
```

```

90     mu_diameter,
91     sigma_diameter
92 ):
93     """Generate a distribution of ice floes that cover a given concentration of
94     ↪ the surface."""
95     total_ice_area = ice_concentration * area_domain
96
97     current_ice_area = 0.0
98
99     while current_ice_area < total_ice_area:
100
101         diameter = np.random.lognormal(mu_diameter, sigma_diameter)
102
103         if diameter > max_diameter or diameter < min_diameter:
104             continue
105
106         radius = diameter / 2
107         area_floe = np.pi * radius ** 2
108
109         ice_floes.append({
110             "diameter": diameter,
111             "area": area_floe
112         })
113
114         current_ice_area += area_floe
115
116     return ice_floes, current_ice_area
117
118 ice_floes, current_ice_area = ice_floes_diameter_distribution(
119     ice_floes,
120     ice_concentration,
121     area_domain,
122     mu_diameter,
123     sigma_diameter
124 )
125
126 print(
127     f"Generated {len(ice_floes)} ice floes with a total area of "
128     f"{current_ice_area:.2f} m2, covering {current_ice_area / area_domain:.2%} "
129     f"of the surface."
130 )
131
132 ice_floes = sorted(ice_floes, key=lambda x: x["diameter"], reverse=True)
133
134
135 # =====
136 # PLACE ICE FLOES WITHOUT OVERLAP
137 # =====

```

```
138
139 placed_floes = []
140
141 for i, floe in enumerate(ice_floes):
142
143     diameter = floe["diameter"]
144     print(f"Diameter of floe {i}: {diameter:.2f} m")
145
146     radius = diameter / 2
147     placed = False
148
149     while placed is False:
150
151         x = np.random.uniform(coords[0][0] + radius, coords[1][0] - radius)
152         y = np.random.uniform(coords[0][1] + radius, coords[1][1] - radius)
153
154         overlap = False
155
156         for placed_floe in placed_floes:
157
158             dx = x - placed_floe["x"]
159             dy = y - placed_floe["y"]
160             distance = np.sqrt(dx**2 + dy**2)
161
162             if distance < (radius + placed_floe["radius"]):
163                 overlap = True
164                 break
165
166             elif (
167                 (x - radius < coords[0][0])
168                 or (x + radius > coords[1][0])
169                 or (y - radius < coords[0][1])
170                 or (y + radius > coords[1][1])
171             ):
172                 overlap = True
173                 break
174
175         if not overlap:
176             placed_floes.append({
177                 "x": x,
178                 "y": y,
179                 "radius": radius
180             })
181             placed = True
182             break
183
184     if not placed:
185         print(f"Could not place floe with diameter {diameter:.2f} m.")
186
```

```
187
188 # =====
189 # SAVE RAW ICE FLOE FIELD
190 # =====
191
192 raw_field_df = pd.DataFrame([
193     {"x": f["x"], "y": f["y"], "radius": f["radius"]}
194     for f in placed_floes
195 ])
196 raw_field_path = OUTPUT_DIR / "ice_floe_field_raw.csv"
197 raw_field_df.to_csv(raw_field_path, index=False)
198 print(f"\nRaw ice floe field saved to: {raw_field_path} ({len(placed_floes)}
199 ↪ floes)")
200
201 # =====
202 # PLOTTING
203 # =====
204
205 def _plot_field(placed_floes, coords, title=None, filepath=None, show=True):
206     xmin, ymin = coords[0]
207     xmax, ymax = coords[1]
208     aspect = (xmax - xmin) / (ymax - ymin)
209     fig, ax = plt.subplots(figsize=(min(24, aspect * 4), 4))
210
211     for floe in placed_floes:
212         ax.add_patch(Circle(
213             (floe["x"], floe["y"]), floe["radius"],
214             edgecolor="steelblue", facecolor="lightblue",
215             linewidth=0.25, alpha=0.7
216         ))
217
218     ax.add_patch(Rectangle(
219         (xmin, ymin), xmax - xmin, ymax - ymin,
220         fill=False, edgecolor="red", linewidth=2
221     ))
222
223     ax.set_xlim(xmin - 0.1, xmax + 0.1)
224     ax.set_ylim(ymin - 0.1, ymax + 0.1)
225     ax.set_aspect("equal", adjustable="box")
226     ax.set_xlabel("x [m]")
227     ax.set_ylabel("y [m]")
228     ax.set_title(title or f"Ice floe field - {len(placed_floes)} floes")
229     ax.grid(True, linestyle="--", alpha=0.4)
230     plt.tight_layout()
231
232     if filepath:
233         fig.savefig(filepath, dpi=100, bbox_inches="tight")
234         print(f" Saved: {filepath}")
```

```

235     if show:
236         plt.show()
237     else:
238         plt.close(fig)
239
240
241 def _plot_histogram(placed_floes, mu, sigma, min_d, max_d, filepath=None):
242     diameters = [f["radius"] * 2 for f in placed_floes]
243     d_range = np.linspace(min_d, max_d, 300)
244     d_trunc = np.linspace(min_d, max_d, 3000)
245
246     def lognorm_pdf(x):
247         return (1.0 / (x * sigma * np.sqrt(2 * np.pi)))
248             * np.exp(-((np.log(x) - mu) ** 2) / (2 * sigma ** 2)))
249
250     norm = np.trapz(lognorm_pdf(d_trunc), d_trunc)
251     pdf_norm = np.where(
252         (d_range >= min_d) & (d_range <= max_d),
253         lognorm_pdf(d_range) / norm, 0.0
254     )
255
256     fig, ax = plt.subplots(figsize=(8, 4))
257     ax.hist(diameters, bins=30, density=True, alpha=0.6,
258            edgecolor="black", label="Placed floes")
259     ax.plot(d_range, pdf_norm, linewidth=2, label="Truncated log-normal PDF")
260     ax.axvline(min_d, linestyle="--", linewidth=1.2)
261     ax.axvline(max_d, linestyle="--", linewidth=1.2, label="Min / Max diameter")
262     ax.set_xlabel("Diameter [m]")
263     ax.set_ylabel("Probability density")
264     ax.legend()
265     ax.grid(True, linestyle="--", alpha=0.4)
266     plt.tight_layout()
267
268     if filepath:
269         fig.savefig(filepath, dpi=150, bbox_inches="tight")
270         print(f" Saved: {filepath}")
271     plt.show()
272
273
274 c_tag = f"C{int(ice_concentration * 100)}"
275
276 _plot_field(
277     placed_floes, coords,
278     title = f"Ice floe field - C = {ice_concentration:.0%}
279     ↪ ({len(placed_floes)} floes)",
280     filepath = OUTPUT_DIR / f"ice_floe_field_{c_tag}.png",
281     show = False,
282 )

```

```

283 _plot_histogram(
284     placed_floes, mu_diameter, sigma_diameter,
285     min_diameter, max_diameter,
286     filepath = OUTPUT_DIR / f"diameter_histogram_{c_tag}.png",
287 )
288
289
290 # =====
291 # FUNCTION TO EXPORT ONE CSV PER VELOCITY
292 # =====
293
294 def save_injection_csv_for_velocity(
295     velocity,
296     placed_floes,
297     coords,
298     ice_concentration,
299     thickness,
300     duration_impact,
301     time_first_impact,
302     time_step,
303     x_injection,
304     Lpp,
305     g,
306     output_dir,
307 ):
308     """
309     Export one injection CSV for a given velocity.
310
311     The large ice field has already been generated.
312     For each velocity, only the portion corresponding to duration_impact is
313     ↪ used.
314     """
315
316     print()
317     print(f"=== Export CSV for velocity = {velocity:.3f} m/s ===")
318
319     start_injection_time = time_first_impact - (x_injection - Lpp) / velocity
320     injection_end_time = start_injection_time + duration_impact
321
322     Fr = np.sqrt(velocity**2 / (g * Lpp))
323
324     xmin, ymin = coords[0]
325     xmax, ymax = coords[1]
326
327     domain_length = xmax - xmin
328
329     extended_floes = sorted(placed_floes, key=lambda x: x["x"])
330
331     print(f"start_injection_time = {start_injection_time:.3f} s")

```

```

331     print(f"injection_end_time   = {injection_end_time:.3f} s")
332     print(f"injection duration   = {duration_impact:.3f} s")
333     print(f"Froude number        = {Fr:.4f}")
334     print(f"Number of floes in large field: {len(extended_floes)}")
335
336     t_bunch = 50 * time_step
337     l_bunch = velocity * t_bunch
338
339     print(f"t_bunch = {t_bunch:.6f} s")
340     print(f"l_bunch = {l_bunch:.6f} m")
341
342     injection_data_bunch = []
343
344     current_time = start_injection_time + t_bunch
345
346     z_equilibrium_cylinder = 0.0
347
348     while current_time <= injection_end_time + 1e-12:
349
350         x_min_bunch = (
351             velocity * (current_time - start_injection_time - t_bunch)
352             + xmin
353         )
354
355         x_max_bunch = (
356             velocity * (current_time - start_injection_time)
357             + xmin
358         )
359
360         for floe in extended_floes:
361
362             if x_min_bunch < floe["x"] <= x_max_bunch:
363
364                 x_floe_adjusted = (
365                     floe["x"]
366                     - velocity * (current_time - start_injection_time)
367                     + domain_length
368                 )
369
370                 injection_data_bunch.append({
371                     "time": round(current_time - t_bunch, 10),
372                     "x": x_floe_adjusted,
373                     "y": floe["y"],
374                     "z": z_equilibrium_cylinder,
375                     "Radius": floe["radius"],
376                     "thickness": thickness
377                 })
378
379         current_time += t_bunch

```

```

380
381 injection_bunch_df = pd.DataFrame(injection_data_bunch)
382
383 velocity_str = f"{velocity:.2f}".replace(".", "p")
384 Fr_str = f"{Fr:.3f}".replace(".", "p")
385
386 filename = (
387     f"C{int(ice_concentration * 100)}"
388     f"_V{velocity_str}mps"
389     f"_Fr{Fr_str}"
390     f"_t{int(thickness * 1000)}mm"
391     f"_tinject{start_injection_time:.2f}s".replace(".", "p")
392     + f"_duration{int(duration_impact)}s"
393     f".csv"
394 )
395
396 filepath = output_dir / filename
397
398 injection_bunch_df.to_csv(filepath, index=False)
399
400 print(f"Bunch injection data saved to: {filepath}")
401 print(f"Number of injected floes: {len(injection_bunch_df)}")
402
403 if len(injection_bunch_df) > 0:
404     print(injection_bunch_df.head())
405 else:
406     print("Warning: no floes injected for this velocity.")
407
408 return injection_bunch_df, filepath
409
410
411 # =====
412 # EXPORT ONE CSV FOR EACH VELOCITY
413 # =====
414
415 all_output_files = []
416
417 for velocity_i in velocities:
418
419     injection_df, filepath = save_injection_csv_for_velocity(
420         velocity=velocity_i,
421         placed_floes=placed_floes,
422         coords=coords,
423         ice_concentration=ice_concentration,
424         thickness=thickness,
425         duration_impact=duration_impact,
426         time_first_impact=time_first_impact,
427         time_step=time_step,
428         x_injection=x_injection,

```

```
429     Lpp=Lpp,
430     g=g,
431     output_dir=OUTPUT_DIR,
432 )
433
434 all_output_files.append({
435     "velocity_mps": velocity_i,
436     "n_injected_floes": len(injection_df),
437     "filepath": str(filepath)
438 })
439
440
441 # =====
442 # SAVE SUMMARY
443 # =====
444
445 summary_df = pd.DataFrame(all_output_files)
446 summary_filepath = OUTPUT_DIR / "injection_files_summary.csv"
447 summary_df.to_csv(summary_filepath, index=False)
448
449 print()
450 print("All CSV files generated.")
451 print(f"Summary saved to: {summary_filepath}")
452 print(summary_df)
453
454 elapsed = time.perf_counter() - t_start
455 print(f"\nTotal run time : {int(elapsed // 60)} min {elapsed % 60:.1f} s")
456
457 plt.show()
```

A.2 Ice Floe field progressive growth algorithm.

See Section 3.5.3. This second algorithm is based on a progressive-growth strategy. This algorithm is really powerful to reach high ice concentration ($c_{ice} = 85\%$, maybe more), but take more time to generate.

For example, the ice floe field generated for the KCS (40s at maximum velocity 1.2m/s) took 79 min.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from matplotlib.patches import Circle, Rectangle
5 from pathlib import Path
6 from scipy.spatial import cKDTree
7 import time
8
9
10 # =====
11 # PARAMETERS - edit this section only
12 # =====
13
14
15 # Physics
16 rho_water = 997.561
17 rho_ice = 900.0
18 g = 9.81
19
20 # Ship
21 Lpp = 4.367
22
23 # Injection geometry
24 x_injection = 10.0 # x-position of the injection plane [m]
25 y_injection = 2.5 # domain half-width in y [m]
26
27 # Velocities to generate injection CSVs for
28 velocities = np.array([0.4, 0.6, 0.8, 1.0, 1.2]) # [m/s]
29
30 # Timing
31 time_step = 0.01 # CFD time step [s]
32 time_first_impact = 40.0 # time when ice first reaches the ship bow [s]
33 duration_impact = 20.0 # duration of the ice injection phase [s]
34
35 # Floe geometry
36 thickness = 0.02 # ice thickness [m]
37 ice_concentration = 0.50 # target ice concentration (0-1)
38
39 # Minimum gap between floes
40 # Guarantees that every pair of floes satisfies:
41 # dist(i, j) >= R_i + R_j + min_gap
42 # The packing internally inflates each radius by min_gap/2 (effective radius).
```

```

43 # The exported CSV always contains the real (un-inflated) radii.
44 # Set to 0.0 to allow floes to touch (original behaviour).
45 min_gap = 0.003 # [m] e.g. 2 mm
46
47 # Size distribution - truncated log-normal on diameters
48 min_diameter = 0.05 # [m]
49 max_diameter = 0.40 # [m]
50 mu_diameter = -1.59 # mean of ln(diameter)
51 sigma_diameter = 0.45 # std of ln(diameter)
52
53 # Reproducibility
54 random_seed = 17
55 rng = np.random.default_rng(random_seed)
56
57 # Progressive-growth tuning
58 initial_radius_scale = 0.40 # start floes at this fraction of effective
    ↪ radius
59 initial_growth_step = 0.04
60 minimum_growth_step = 0.001
61 growth_step_increase = 1.15
62 growth_step_decrease = 0.50
63
64 # Relaxation
65 relaxation_steps_per_growth = 2000
66 final_relaxation_steps = 12000
67 relaxation_factor = 0.45
68 target_max_overlap = 1e-6 # [m] - in effective-radius space
69
70 # Initial placement
71 # Best-clearance strategy: test n_candidates random positions per floe
72 # and keep the one with maximum clearance from already-placed floes.
73 n_candidates_initial = 2000
74
75 # Relocation (escape from stuck configurations)
76 enable_relocation = True
77 relocation_fraction = 0.10
78 relocation_candidates = 2000
79
80 # Final shrink
81 enable_final_shrink = True
82 tiny_overlap_threshold = 1e-6
83 final_shrink_safety = 1e-12
84
85 # Intermediate growth plots
86 save_growth_plots = False
87 plot_every_n_steps = 10
88
89
90 BASE_DIR = Path(__file__).resolve().parent / "output_IF_progressive_growth"

```

```

91 OUTPUT_DIR = BASE_DIR / rf"C{int(ice_concentration * 100)}"
92 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
93 PLOT_DIR = BASE_DIR / "plots_growth"
94 PLOT_DIR.mkdir(parents=True, exist_ok=True)
95
96
97 # =====
98 # DOMAIN SIZING (based on max velocity and injection duration)
99 # =====
100
101 max_velocity = float(np.max(velocities))
102 x_output = x_injection - duration_impact * max_velocity # leftmost domain
103 ↪ edge
104
105 coords = np.array([
106     [x_output, -y_injection],
107     [x_injection, y_injection],
108 ])
109
110 # KDTree interaction radius: two effective discs can interact if
111 # dist < (R_eff_i + R_eff_j)_max = max_effective_diameter
112 # where R_eff = R + min_gap/2 max_effective_diameter = max_diameter + min_gap
113 _MAX_INTERACT = max_diameter + min_gap
114
115 print("=" * 60)
116 print("Ice floe generator - progressive-growth packing")
117 print(f" Target concentration : {ice_concentration:.0%}")
118 print(f" Minimum gap : {min_gap * 1000:.1f} mm")
119 print(f" Velocities : {velocities} m/s")
120 print(f" Domain x : [{x_output:.3f}, {x_injection}] m")
121 print(f" Domain y : [{-y_injection}, {y_injection}] m")
122 print(f" Domain area : {(x_injection - x_output) * 2 *
123 ↪ y_injection:.3f} m²")
124 print("=" * 60)
125 print()
126
127 # =====
128 # GEOMETRY HELPERS
129 # =====
130
131 def domain_bounds(coords):
132     (xmin, ymin), (xmax, ymax) = coords
133     return xmin, ymin, xmax, ymax
134
135 def domain_area(coords):
136     xmin, ymin, xmax, ymax = domain_bounds(coords)
137     return (xmax - xmin) * (ymax - ymin)

```

```

138
139
140 def achieved_concentration(radii, coords):
141     """Ice concentration based on actual (non-inflated) radii."""
142     if len(radii) == 0:
143         return 0.0
144     return float(np.sum(np.pi * np.asarray(radii) ** 2) / domain_area(coords))
145
146
147 def clamp_to_domain(positions, radii, coords):
148     """Keep every floe centre at least one radius away from each wall."""
149     xmin, ymin, xmax, ymax = domain_bounds(coords)
150     positions[:, 0] = np.clip(positions[:, 0], xmin + radii, xmax - radii)
151     positions[:, 1] = np.clip(positions[:, 1], ymin + radii, ymax - radii)
152     return positions
153
154
155 # =====
156 # OVERLAP HELPERS (KDTree-accelerated)
157 # NOTE: pass *effective* radii (R + min_gap/2) to enforce the gap.
158 # =====
159
160 def _overlapping_pairs(positions, eff_radii):
161     """
162     Return (ia, ja, overlaps) for all pairs where
163     dist(i, j) < eff_R_i + eff_R_j
164     i.e. where the effective-radius discs overlap.
165     """
166     if len(eff_radii) < 2:
167         empty = np.array([], dtype=int)
168         return empty, empty, np.array([])
169
170     tree = cKDTree(positions)
171     pairs = tree.query_pairs(_MAX_INTERACT)
172     if not pairs:
173         empty = np.array([], dtype=int)
174         return empty, empty, np.array([])
175
176     ia = np.fromiter((p[0] for p in pairs), dtype=int, count=len(pairs))
177     ja = np.fromiter((p[1] for p in pairs), dtype=int, count=len(pairs))
178     dx = positions[ia, 0] - positions[ja, 0]
179     dy = positions[ia, 1] - positions[ja, 1]
180     dist = np.sqrt(dx**2 + dy**2)
181     ov = eff_radii[ia] + eff_radii[ja] - dist
182     mask = ov > 0
183     return ia[mask], ja[mask], ov[mask]
184
185
186 def overlap_statistics(positions, eff_radii):

```

```

187     """Return (n_overlaps, max_overlap, penalty, score_per_floe)."""
188     ia, ja, ov = _overlapping_pairs(positions, eff_radii)
189     n_ov = len(ov)
190     max_ov = float(ov.max()) if n_ov > 0 else 0.0
191     penalty = float(np.sum(ov ** 2))
192     score = np.zeros(len(eff_radii))
193     if n_ov > 0:
194         np.add.at(score, ia, ov ** 2)
195         np.add.at(score, ja, ov ** 2)
196     return n_ov, max_ov, penalty, score
197
198
199 # =====
200 # MIN-GAP VERIFICATION
201 # =====
202
203 def check_min_gap(positions, actual_radii, min_gap, tol=1e-9):
204     """
205     Verify that every pair of floes satisfies:
206     dist(i, j) >= R_i + R_j + min_gap (up to tol)
207
208     Returns
209     -----
210     n_violations : int      number of pairs below the required gap
211     min_clearance : float   smallest edge-to-edge clearance found [m]
212     """
213     if min_gap <= 0.0 or len(actual_radii) < 2:
214         return 0, np.inf
215
216     tree = cKDTree(positions)
217     pairs = tree.query_pairs(_MAX_INTERACT)
218
219     n_viol = 0
220     min_clr = np.inf
221     for i, j in pairs:
222         dx = positions[i, 0] - positions[j, 0]
223         dy = positions[i, 1] - positions[j, 1]
224         dist = np.sqrt(dx**2 + dy**2)
225         clr = dist - actual_radii[i] - actual_radii[j]
226         if clr < min_clr:
227             min_clr = clr
228         if clr < min_gap - tol:
229             n_viol += 1
230
231     return n_viol, float(min_clr)
232
233
234 # =====
235 # RELAXATION (KDTree-accelerated, works on effective radii)

```

```

236 # =====
237
238 def relax_overlaps(positions, eff_radii, coords, rng,
239                   n_steps=1000, factor=0.45,
240                   target_max_overlap=1e-6, verbose=False):
241     """
242     Soft-disc relaxation on effective radii.
243     Overlapping pairs push each other apart proportionally to their
244     inverse mass (smaller floes move more).
245     """
246     positions = positions.copy()
247     eff_radii = np.asarray(eff_radii)
248     eps      = 1e-12
249     inv_mass  = 1.0 / (eff_radii ** 2 + eps)
250
251     for step in range(n_steps):
252         tree = cKDTree(positions)
253         pairs = tree.query_pairs(_MAX_INTERACT)
254         if not pairs:
255             break
256
257         ia = np.fromiter((p[0] for p in pairs), dtype=int, count=len(pairs))
258         ja = np.fromiter((p[1] for p in pairs), dtype=int, count=len(pairs))
259         dx = positions[ia, 0] - positions[ja, 0]
260         dy = positions[ia, 1] - positions[ja, 1]
261         dist = np.sqrt(dx**2 + dy**2)
262         ov = eff_radii[ia] + eff_radii[ja] - dist
263         mask = ov > 0
264         n_ov = int(mask.sum())
265         max_ov = float(ov[mask].max()) if n_ov > 0 else 0.0
266
267         if verbose and step % 200 == 0:
268             pen = float(np.sum(ov[mask] ** 2)) if n_ov > 0 else 0.0
269             print(f"    step {step:5d} | overlaps {n_ov:5d} | "
270                   f"max_ov {max_ov:.3e} m | pen {pen:.3e}")
271
272         if n_ov == 0 or max_ov <= target_max_overlap:
273             break
274
275         ia_m = ia[mask]; ja_m = ja[mask]
276         ov_m = ov[mask]
277         dx_m = dx[mask]; dy_m = dy[mask]
278         dist_m = dist[mask]
279
280         tiny = dist_m < eps
281         rand_angles = rng.uniform(0, 2 * np.pi, int(tiny.sum()))
282         nx = np.where(tiny,
283                      np.cos(rand_angles[np.cumsum(tiny) - 1]) if
284                      rand_angles.size else 0.0,

```

```

284         dx_m / np.where(tiny, 1.0, dist_m))
285     ny = np.where(tiny,
286                   np.sin(rand_angles[np.cumsum(tiny) - 1]) if
287                   ↪ rand_angles.size else 0.0,
288                   dy_m / np.where(tiny, 1.0, dist_m))
289
290     inv_sum = inv_mass[ia_m] + inv_mass[ja_m]
291     si      = inv_mass[ia_m] / inv_sum
292     sj      = inv_mass[ja_m] / inv_sum
293     corr    = factor * ov_m
294
295     disp_x = np.zeros(len(eff_radii))
296     disp_y = np.zeros(len(eff_radii))
297     np.add.at(disp_x, ia_m, corr * si * nx)
298     np.add.at(disp_y, ia_m, corr * si * ny)
299     np.add.at(disp_x, ja_m, -corr * sj * nx)
300     np.add.at(disp_y, ja_m, -corr * sj * ny)
301
302     positions[:, 0] += disp_x
303     positions[:, 1] += disp_y
304     positions = clamp_to_domain(positions, eff_radii, coords)
305
306     return positions
307
308 # =====
309 # STEP 1 - GENERATE RADII
310 # =====
311
312 def generate_radii(coords, target_concentration, mu, sigma,
313                  min_d, max_d, rng):
314     """
315     Draw diameters from a truncated log-normal distribution until the
316     cumulative actual ice area >= target_concentration * domain_area.
317     Returns actual (non-inflated) radii sorted largest-first.
318     """
319     target_area = target_concentration * domain_area(coords)
320     radii      = []
321     current_area = 0.0
322
323     while current_area < target_area:
324         d = rng.lognormal(mu, sigma)
325         if d < min_d or d > max_d:
326             continue
327         r = 0.5 * d
328         radii.append(r)
329         current_area += np.pi * r ** 2
330
331     radii = np.sort(np.array(radii))[:, :-1]

```

```

332
333     print("Diameter generation")
334     print("-----")
335     print(f"   Floes generated           : {len(radii)}")
336     print(f"   Target concentration       : {target_concentration:.2%}")
337     print(f"   Achieved concentration      : {achieved_concentration(radii,
338     ↪   coords):.2%}")
339     print(f"   Diameter range              : {2 * radii[-1]:.3f} - {2 * radii[0]:.3f}
340     ↪   m")
341     print(f"   Mean diameter              : {2 * np.mean(radii):.3f} m")
342     print()
343     return radii
344
345 # =====
346 # STEP 2 - INITIAL PLACEMENT (best-clearance + KDTree)
347 # =====
348
349 def place_initial_shrunk_floes(eff_final_radii, coords, rng,
350     initial_scale=0.40,
351     n_candidates=2000):
352     """
353     Place floes at `initial_scale * eff_final_radius` using a
354     best-clearance strategy (from IF_generator_algoVI_domain_star.py)
355     accelerated by a KDTree.
356
357     For each floe, `n_candidates` random positions are tested and the one
358     with maximum clearance from already-placed floes is kept. If a
359     position with strictly positive clearance is found it is used
360     immediately without testing the remaining candidates.
361     """
362     xmin, ymin, xmax, ymax = domain_bounds(coords)
363     working_eff_radii = initial_scale * eff_final_radii
364     n = len(working_eff_radii)
365     positions = np.zeros((n, 2))
366
367     tree = None
368     tree_built_at = 0
369     rebuild_every = 100
370
371     for k, r in enumerate(working_eff_radii):
372         # Rebuild KDTree periodically
373         if k > 0 and (k - tree_built_at) >= rebuild_every:
374             tree = cKDTree(positions[:k])
375             tree_built_at = k
376
377         best_pos = np.array([rng.uniform(xmin + r, xmax - r),
378             rng.uniform(ymin + r, ymax - r)])

```

```

379     best_clr = -np.inf
380
381     for _ in range(n_candidates):
382         cand = np.array([rng.uniform(xmin + r, xmax - r),
383                          rng.uniform(ymin + r, ymax - r)])
384
385         if k == 0:
386             best_pos = cand
387             best_clr = np.inf
388             break
389
390         # Fast neighbour lookup
391         if tree is not None:
392             nearby = tree.query_ball_point(cand, _MAX_INTERACT *
393                                           ↪ initial_scale + r)
394         else:
395             nearby = list(range(k))
396
397         if not nearby:
398             # No neighbours → perfect spot, stop immediately
399             best_pos = cand
400             best_clr = np.inf
401             break
402
403         pos_n = positions[np.array(nearby)]
404         r_n = working_eff_radrii[np.array(nearby)]
405         dists = np.sqrt(((pos_n[:, 0] - cand[0]) ** 2
406                          + (pos_n[:, 1] - cand[1]) ** 2))
407         clr = float(np.min(dists - r_n - r))
408
409         if clr > best_clr:
410             best_clr = clr
411             best_pos = cand
412
413     positions[k] = best_pos
414
415     if (k + 1) % 500 == 0 or k + 1 == n:
416         c = achieved_concentration(working_eff_radrii[:k + 1], coords)
417         print(f" Placement: {k+1:5d}/{n} floes | "
418               ↪ f"C_eff = {c:.1%} | best_clr = {best_clr:.3e} m")
419
420     n_ov, max_ov, _, _ = overlap_statistics(positions, working_eff_radrii)
421     print()
422     print("Initial placement complete")
423     print(f" Radius scale : {initial_scale:.3f}")
424     print(f" C_eff : {achieved_concentration(working_eff_radrii,
425     ↪ coords):.2%}")
426     print(f" Overlaps : {n_ov} max = {max_ov:.3e} m")
427     print()

```

```
426     return positions
427
428
429 # =====
430 # STEP 3 - RELOCATION (escape stuck configurations)
431 # =====
432
433 def relocate_problematic_floes(positions, eff_radai, coords, rng,
434                                fraction=0.10, n_candidates=2000):
435     """
436     Move the most-overlapping floes to better positions (best-clearance
437     among n_candidates random spots, KDTree-accelerated).
438     """
439     positions = positions.copy()
440     xmin, ymin, xmax, ymax = domain_bounds(coords)
441
442     _, _, _, score = overlap_statistics(positions, eff_radai)
443     n_move = max(1, int(fraction * len(eff_radai)))
444     problematic = np.where(score > 0)[0]
445     if len(problematic) == 0:
446         return positions
447
448     ranking = score[problematic] / (eff_radai[problematic] ** 2 + 1e-12)
449     order = np.argsort(ranking)[::-1]
450     selected = problematic[order[:min(n_move, len(problematic))]]
451
452     tree = cKDTree(positions)
453
454     for i in selected:
455         r = eff_radai[i]
456
457         # Current clearance
458         nearby_cur = tree.query_ball_point(positions[i], _MAX_INTERACT + r)
459         if i in nearby_cur:
460             nearby_cur.remove(i)
461         if nearby_cur:
462             pos_n = positions[np.array(nearby_cur)]
463             r_n = eff_radai[np.array(nearby_cur)]
464             dists = np.sqrt(((pos_n[:, 0] - positions[i, 0]) ** 2
465                             + (pos_n[:, 1] - positions[i, 1]) ** 2))
466             best_clr = float(np.min(dists - r_n - r))
467         else:
468             best_clr = np.inf
469         best_pos = positions[i].copy()
470
471         for _ in range(n_candidates):
472             cand = np.array([rng.uniform(xmin + r, xmax - r),
473                             rng.uniform(ymin + r, ymax - r)])
474             nearby = tree.query_ball_point(cand, _MAX_INTERACT + r)
```

```
475         if i in nearby:
476             nearby.remove(i)
477         if not nearby:
478             best_pos = cand
479             break
480         pos_n = positions[np.array(nearby)]
481         r_n = eff_radii[np.array(nearby)]
482         dists = np.sqrt(((pos_n[:, 0] - cand[0]) ** 2
483                        + (pos_n[:, 1] - cand[1]) ** 2))
484         clr = float(np.min(dists - r_n - r))
485         if clr > best_clr:
486             best_clr = clr
487             best_pos = cand
488
489         positions[i] = best_pos
490
491     return positions
492
493
494 # =====
495 # STEP 4 - PROGRESSIVE GROWTH MAIN LOOP
496 # =====
497
498 def progressive_growth_packing(
499     eff_final_radii, coords, rng,
500     initial_scale=0.40, initial_growth_step=0.04,
501     minimum_growth_step=0.001, growth_step_increase=1.15,
502     growth_step_decrease=0.50, relax_steps=2000,
503     final_relax_steps=12000, relax_factor=0.45,
504     target_max_overlap=1e-6, enable_relocation=True,
505     reloc_fraction=0.10, reloc_candidates=2000,
506     n_cand_initial=2000, save_plots=False, plot_every=10,
507 ):
508     """
509     Progressive-growth packing on effective radii.
510
511     Grows floes from `initial_scale * eff_final_radii` to `eff_final_radii`
512     in adaptive steps, relaxing overlaps after each step. Returns
513     (positions, eff_final_radii_sorted).
514     """
515     eff_final_radii = np.sort(np.asarray(eff_final_radii))[:, -1]
516
517     positions = place_initial_shrunk_floes(
518         eff_final_radii, coords, rng,
519         initial_scale=initial_scale,
520         n_candidates=n_cand_initial,
521     )
522
523     scale = initial_scale
```

```
524 growth_step = initial_growth_step
525 current_eff_r = scale * eff_final_radai
526 growth_id = 0
527
528 # Initial relaxation
529 positions = relax_overlaps(
530     positions, current_eff_r, coords, rng,
531     n_steps=relax_steps, factor=relax_factor,
532     target_max_overlap=target_max_overlap,
533 )
534
535 while scale < 1.0 - 1e-12:
536     trial_scale = min(1.0, scale + growth_step)
537     trial_eff_r = trial_scale * eff_final_radai
538
539     print(f"\nGrowth {growth_id:4d} | "
540           f"scale {scale:.4f} -> {trial_scale:.4f} | "
541           f"C_eff = {achieved_concentration(trial_eff_r, coords):.2%} | "
542           f"step = {growth_step:.4f}")
543
544     trial_pos = relax_overlaps(
545         positions.copy(), trial_eff_r, coords, rng,
546         n_steps=relax_steps, factor=relax_factor,
547         target_max_overlap=target_max_overlap,
548     )
549
550     n_ov, max_ov, pen, _ = overlap_statistics(trial_pos, trial_eff_r)
551     print(f" After relax | overlaps {n_ov:5d} | "
552           f"max_ov {max_ov:.3e} m | pen {pen:.3e}")
553
554     success = (n_ov == 0 or max_ov <= target_max_overlap)
555
556     if not success and enable_relocation:
557         print(" Relocating stuck floes...")
558         trial_pos = relocate_problematic_floes(
559             trial_pos, trial_eff_r, coords, rng,
560             fraction=reloc_fraction, n_candidates=reloc_candidates,
561         )
562         trial_pos = relax_overlaps(
563             trial_pos, trial_eff_r, coords, rng,
564             n_steps=relax_steps, factor=relax_factor,
565             target_max_overlap=target_max_overlap,
566         )
567         n_ov, max_ov, pen, _ = overlap_statistics(trial_pos, trial_eff_r)
568         print(f" After reloc | overlaps {n_ov:5d} | "
569               f"max_ov {max_ov:.3e} m | pen {pen:.3e}")
570         success = (n_ov == 0 or max_ov <= target_max_overlap)
571
572     if success:
```

```

573     positions      = trial_pos
574     scale          = trial_scale
575     current_eff_r  = trial_eff_r
576     growth_step    = min(growth_step * growth_step_increase, 0.05)
577     print(" Growth accepted.")
578
579     if save_plots and growth_id % plot_every == 0:
580         _plot_field(
581             positions, current_eff_r, coords,
582             title=(f"Growth {growth_id} - scale {scale:.3f} - "
583                   f"C_eff = {achieved_concentration(current_eff_r,
584                                                       ↪ coords):.1%}"),
585             filepath=PLOT_DIR / f"growth_{growth_id:04d}.png",
586             show=False,
587         )
588     else:
589         growth_step *= growth_step_decrease
590         print(f" Growth rejected. New step = {growth_step:.4f}")
591         if growth_step < minimum_growth_step:
592             print(" Minimum growth step reached - accepting and exiting.")
593             positions      = trial_pos
594             scale          = trial_scale
595             current_eff_r  = trial_eff_r
596             break
597
598     growth_id += 1
599
600     # Final relaxation
601     print()
602     print("Final relaxation")
603     print("-----")
604     positions = relax_overlaps(
605         positions, eff_final_radII, coords, rng,
606         n_steps=final_relax_steps, factor=relax_factor,
607         target_max_overlap=target_max_overlap, verbose=True,
608     )
609
610     # Final relocation passes
611     if enable_relocation:
612         _, max_ov, _, _ = overlap_statistics(positions, eff_final_radII)
613         if max_ov > target_max_overlap:
614             print()
615             print("Final relocation passes")
616             print("-----")
617             for pass_id in range(5):
618                 positions = relocate_problematic_floes(
619                     positions, eff_final_radII, coords, rng,
620                     fraction=reloc_fraction, n_candidates=reloc_candidates,

```

```

621         positions = relax_overlaps(
622             positions, eff_final_radii, coords, rng,
623             n_steps=final_relax_steps // 2, factor=relax_factor,
624             target_max_overlap=target_max_overlap,
625         )
626         n_ov, max_ov, pen, _ = overlap_statistics(positions,
627             ↪ eff_final_radii)
628         print(f" Pass {pass_id + 1} | overlaps {n_ov} | "
629             f"max_ov {max_ov:.3e} m | pen {pen:.3e}")
630         if n_ov == 0 or max_ov <= target_max_overlap:
631             break
632
633     return positions, eff_final_radii
634
635 # =====
636 # STEP 5 - FINAL TINY-OVERLAP SHRINK
637 # =====
638
639 def final_shrink(positions, eff_radii,
640                 threshold=1e-6, safety=1e-12, enable=True):
641     """
642     If the maximum effective-radius overlap is smaller than `threshold`,
643     shrink every effective radius by (max_overlap / 2 + safety) to
644     guarantee zero overlap. The concentration check uses effective radii
645     so the condition is satisfied whenever actual concentration >= target.
646     """
647     eff_radii = np.asarray(eff_radii).copy()
648     n_ov, max_ov, pen, _ = overlap_statistics(positions, eff_radii)
649
650     print()
651     print("Final shrink check")
652     print("-----")
653     print(f" Overlaps      : {n_ov}      max = {max_ov:.3e} m")
654
655     if not enable:
656         print(" Disabled.")
657         return eff_radii, 0.0, n_ov, max_ov, pen
658
659     if n_ov > 0 and max_ov < threshold:
660         shrink = 0.5 * max_ov + safety
661         eff_radii -= shrink
662         n_ov2, max_ov2, pen2, _ = overlap_statistics(positions, eff_radii)
663         print(f" Shrink applied : {shrink:.3e} m")
664         print(f" Overlaps left  : {n_ov2}")
665         return eff_radii, shrink, n_ov2, max_ov2, pen2
666
667     print(" No shrink needed." if n_ov == 0 else
668         " Max overlap >= threshold - no shrink applied.")

```

```

669     return eff_radii, 0.0, n_ov, max_ov, pen
670
671
672 # =====
673 # PLOTTING
674 # =====
675
676 def _plot_field(positions, radii, coords, title=None, filepath=None, show=True):
677     xmin, ymin, xmax, ymax = domain_bounds(coords)
678     aspect = (xmax - xmin) / (ymax - ymin)
679     fig, ax = plt.subplots(figsize=(min(24, aspect * 4), 4))
680
681     for (px, py), r in zip(positions, radii):
682         ax.add_patch(Circle((px, py), r,
683                             edgecolor="steelblue", facecolor="lightblue",
684                             linewidth=0.25, alpha=0.7))
685
686     ax.add_patch(Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
687                           fill=False, edgecolor="red", linewidth=2))
688
689     c = achieved_concentration(radii, coords)
690     ax.set_xlim(xmin - 0.5, xmax + 0.5)
691     ax.set_ylim(ymin - 0.3, ymax + 0.3)
692     ax.set_aspect("equal")
693     ax.set_xlabel("x [m]")
694     ax.set_ylabel("y [m]")
695     ax.set_title(title or f"Ice floe field - C = {c:.1%} ({len(radii)} floes)")
696     ax.grid(True, linestyle="--", alpha=0.4)
697     plt.tight_layout()
698
699     if filepath:
700         fig.savefig(filepath, dpi=100, bbox_inches="tight")
701         print(f" Saved: {filepath}")
702     if show:
703         plt.show()
704     else:
705         plt.close(fig)
706
707
708 def _plot_histogram(radii, mu, sigma, min_d, max_d, filepath=None):
709     diameters = 2.0 * np.asarray(radii)
710     d_range = np.linspace(min_d, max_d, 300)
711     d_trunc = np.linspace(min_d, max_d, 3000)
712
713     def lognorm_pdf(x):
714         return (1.0 / (x * sigma * np.sqrt(2 * np.pi)))
715             * np.exp(-((np.log(x) - mu) ** 2) / (2 * sigma ** 2)))
716
717     norm = np.trapz(lognorm_pdf(d_trunc), d_trunc)

```

```

718 pdf_norm = np.where((d_range >= min_d) & (d_range <= max_d),
719                     lognorm_pdf(d_range) / norm, 0.0)
720
721 fig, ax = plt.subplots(figsize=(8, 4))
722 ax.hist(diameters, bins=30, density=True, alpha=0.6,
723         edgecolor="black", label="Placed floes")
724 ax.plot(d_range, pdf_norm, linewidth=2, label="Truncated log-normal PDF")
725 ax.axvline(min_d, linestyle="--", linewidth=1.2)
726 ax.axvline(max_d, linestyle="--", linewidth=1.2, label="Min / Max diameter")
727 ax.set_xlabel("Diameter [m]")
728 ax.set_ylabel("Probability density")
729 ax.legend()
730 ax.grid(True, linestyle="--", alpha=0.4)
731 plt.tight_layout()
732
733 if filepath:
734     fig.savefig(filepath, dpi=150, bbox_inches="tight")
735     print(f" Saved: {filepath}")
736 plt.show()
737
738
739 # =====
740 # INJECTION CSV EXPORT (one per velocity)
741 # =====
742
743 def save_injection_csv(velocity, placed_floes, coords,
744                       ice_concentration, min_gap, thickness,
745                       duration_impact, time_first_impact,
746                       time_step, x_injection, Lpp,
747                       g, output_dir):
748     """
749     Write the injection CSV for one velocity.
750
751     The field is divided into time-bunches of width  $t_{bunch} = 50 * dt$ .
752     At each bunch time, floes whose reference x-coordinate falls within
753     the current bundle window are emitted at the inlet x_injection.
754     """
755     print(f"\n velocity = {velocity:.3f} m/s")
756
757     Fr = velocity / np.sqrt(g * Lpp)
758     t_start = time_first_impact - (x_injection - Lpp) / velocity
759     t_end = t_start + duration_impact
760     t_bunch = 50 * time_step
761     xmin = coords[0, 0]
762     dom_len = coords[1, 0] - xmin
763
764     sorted_floes = sorted(placed_floes, key=lambda f: f["x"])
765
766     print(f" Froude number : {Fr:.4f}")

```

```

767 print(f"    t_inject start    : {t_start:.3f} s")
768 print(f"    t_inject end      : {t_end:.3f} s")
769 print(f"    Floes in field      : {len(sorted_floes)}")
770
771 rows = []
772 current_time = t_start + t_bunch
773
774 while current_time <= t_end + 1e-12:
775     x_win_lo = velocity * (current_time - t_start - t_bunch) + xmin
776     x_win_hi = velocity * (current_time - t_start) + xmin
777
778     for floe in sorted_floes:
779         if x_win_lo < floe["x"] <= x_win_hi:
780             x_adj = floe["x"] - velocity * (current_time - t_start) +
781                 ↪ dom_len
782             rows.append({
783                 "time"      : round(current_time - t_bunch, 10),
784                 "x"         : x_adj,
785                 "y"         : floe["y"],
786                 "z"         : 0.0,
787                 "Radius"    : floe["radius"], # actual (non-inflated)
788                 ↪ radius
789                 "thickness": thickness,
790             })
791
792     current_time += t_bunch
793
794 df = pd.DataFrame(rows)
795
796 v_str = f"{velocity:.2f}".replace(".", "p")
797 Fr_str = f"{Fr:.3f}".replace(".", "p")
798 t_str = f"{t_start:.2f}s".replace(".", "p")
799 gap_str = f"{int(round(min_gap * 1000))}mm"
800 fname = (f"C{int(ice_concentration * 100)}"
801         f"_V{v_str}mps"
802         f"_Fr{Fr_str}"
803         f"_t{int(thickness * 1000)}mm"
804         f"_gap{gap_str}"
805         f"_tinject{t_str}"
806         f"_duration{int(duration_impact)}s.csv")
807
808 fp = output_dir / fname
809 df.to_csv(fp, index=False)
810 print(f"    Saved ({len(df)} rows): {fp.name}")
811 return df, fp
812
813 # =====
814 # SNAPSHOT INJECTION CSV (whole field at once - visual check)

```

```

814 # =====
815
816 def save_snapshot_injection_csv(placed_floes, ice_concentration, min_gap,
817                               thickness, time_first_impact, output_dir):
818     """
819     Dump the entire ice-floe field in a single injection CSV.
820
821     All floes share the same time stamp (time_first_impact) and keep their
822     packed (x, y) coordinates exactly. Injecting this file in STAR-CCM+
823     places every floe simultaneously at t = time_first_impact, which lets
824     you verify the field layout (concentration, size distribution, gaps)
825     before committing to the full time-bunched injection.
826
827     The file is intentionally velocity-independent: it is purely a spatial
828     snapshot of the packed field.
829     """
830     rows = [
831         {
832             "time"      : time_first_impact,
833             "x"         : floe["x"],
834             "y"         : floe["y"],
835             "z"         : 0.0,
836             "Radius"    : floe["radius"],
837             "thickness" : thickness,
838         }
839         for floe in placed_floes
840     ]
841
842     df = pd.DataFrame(rows)
843     gap_str = f"{int(round(min_gap * 1000))}mm"
844     fname = (f"C{int(ice_concentration * 100)}"
845             f"_gap{gap_str}"
846             f"_SNAPSHOT_t{time_first_impact:.2f}s.csv")
847
848     fp = output_dir / fname
849     df.to_csv(fp, index=False)
850     print(f" Snapshot CSV saved ({len(df)} floes at
851           ↪ t={time_first_impact:.2f}s): {fp.name}")
852     return df, fp
853
854 # =====
855 # MAIN
856 # =====
857
858 if __name__ == "__main__":
859
860     t_start = time.perf_counter()
861

```

```

862 # -----
863 # 1. Generate actual radii to reach target concentration
864 # -----
865 final_radii = generate_radii(
866     coords, ice_concentration,
867     mu_diameter, sigma_diameter,
868     min_diameter, max_diameter, rng,
869 )
870
871 # -----
872 # 2. Build effective radii  $R_{eff} = R + min\_gap/2$ 
873 # Packing on  $R_{eff}$  guarantees  $dist(i,j) \geq R_i + R_j + min\_gap$ .
874 # -----
875 eff_final_radii = final_radii + 0.5 * min_gap
876
877 c_actual = achieved_concentration(final_radii, coords)
878 c_eff = achieved_concentration(eff_final_radii, coords)
879 print(f"Actual concentration : {c_actual:.2%}")
880 print(f"Effective concentration : {c_eff:.2%} "
881       f"(packing target - must be below ~0.87)")
882 if c_eff > 0.87:
883     print(" WARNING: effective concentration is very high."
884           " Packing may be slow or fail to converge.")
885 print()
886
887 # -----
888 # 3. Progressive-growth packing on effective radii
889 # -----
890 positions, eff_radii_packed = progressive_growth_packing(
891     eff_final_radii, coords, rng,
892     initial_scale = initial_radius_scale,
893     initial_growth_step = initial_growth_step,
894     minimum_growth_step = minimum_growth_step,
895     growth_step_increase = growth_step_increase,
896     growth_step_decrease = growth_step_decrease,
897     relax_steps = relaxation_steps_per_growth,
898     final_relax_steps = final_relaxation_steps,
899     relax_factor = relaxation_factor,
900     target_max_overlap = target_max_overlap,
901     enable_relocation = enable_relocation,
902     reloc_fraction = relocation_fraction,
903     reloc_candidates = relocation_candidates,
904     n_cand_initial = n_candidates_initial,
905     save_plots = save_growth_plots,
906     plot_every = plot_every_n_steps,
907 )
908
909 # -----
910 # 4. Final tiny-overlap shrink on effective radii

```

```

911 # -----
912 eff_radii_final, shrink, n_ov, max_ov, pen = final_shrink(
913     positions, eff_radii_packed,
914     threshold = tiny_overlap_threshold,
915     safety     = final_shrink_safety,
916     enable     = enable_final_shrink,
917 )
918
919 # -----
920 # 5. Recover actual radii  $R = R_{eff} - min\_gap/2$ 
921 # -----
922 actual_radii = eff_radii_final - 0.5 * min_gap
923 c_final      = achieved_concentration(actual_radii, coords)
924
925 # -----
926 # 6. Verify minimum gap
927 # -----
928 n_viol, min_clr = check_min_gap(positions, actual_radii, min_gap)
929
930 print()
931 print("=" * 60)
932 print("Final result")
933 print("=" * 60)
934 print(f" Target concentration : {ice_concentration:.2%}")
935 print(f" Final concentration   : {c_final:.2%}")
936 print(f" Number of floes        : {len(actual_radii)}")
937 print(f" Eff-radius overlaps     : {n_ov} max = {max_ov:.3e} m")
938 print(f" Radius shrink applied  : {shrink:.3e} m")
939 print()
940 print(f" Min-gap required       : {min_gap * 1000:.2f} mm")
941 print(f" Min clearance found    : {min_clr * 1000:.4f} mm")
942 print(f" Gap violations         : {n_viol}")
943 if n_viol == 0:
944     print(" => Min-gap condition SATISFIED for all pairs.")
945 else:
946     print(" => WARNING: min-gap condition violated for some pairs!")
947 print("=" * 60)
948 print()
949
950 # -----
951 # 7. Save outputs
952 # -----
953 c_tag = f"C{int(round(c_final * 100)):02d}"
954
955 # 7a. Raw geometry CSV (x, y, radius - no time column, for inspection)
956 raw_path = OUTPUT_DIR /
957     ↪ f"ice_floe_field_raw_{c_tag}_gap{int(min_gap*1000)}mm.csv"
958 pd.DataFrame({
959     "x"      : positions[:, 0],

```

```

959         "y"      : positions[:, 1],
960         "radius": actual_radii,
961     }).to_csv(raw_path, index=False)
962     print(f"Raw geometry CSV : {raw_path.name} ({len(actual_radii)} floes)")
963
964     # 7b. Snapshot injection CSV (all floes at t = time_first_impact)
965     #     Use this in STAR-CCM+ to verify the field visually before
966     #     switching to the time-bunched per-velocity files.
967     placed_floes = [
968         {"x": float(positions[i, 0]),
969          "y": float(positions[i, 1]),
970          "radius": float(actual_radii[i])}
971         for i in range(len(actual_radii))
972     ]
973     save_snapshot_injection_csv(
974         placed_floes      = placed_floes,
975         ice_concentration = ice_concentration,
976         min_gap           = min_gap,
977         thickness         = thickness,
978         time_first_impact = time_first_impact,
979         output_dir        = OUTPUT_DIR,
980     )
981
982     # -----
983     # 8. Plots
984     # -----
985     _plot_field(
986         positions, actual_radii, coords,
987         title      = (f"Ice floe field - C = {c_final:.1%} | "
988                      f"{len(actual_radii)} floes | gap >= {min_gap*1000:.0f}
989                      ↪ mm"),
990         filepath   = OUTPUT_DIR /
991                     ↪ f"ice_floe_field_{c_tag}_gap{int(min_gap*1000)}mm.png",
992         show       = False,
993     )
994
995     _plot_histogram(
996         actual_radii, mu_diameter, sigma_diameter,
997         min_diameter, max_diameter,
998         filepath = OUTPUT_DIR / f"diameter_histogram_{c_tag}.png",
999     )
1000
1001     # -----
1002     # 9. Export one time-bunched injection CSV per velocity
1003     # -----
1004     print("\nExporting injection CSVs")
1005     print("-----")
1006     all_files = []
1007     for v in velocities:

```

```
1006     df_inj, fp = save_injection_csv(
1007         velocity      = v,
1008         placed_floes  = placed_floes,
1009         coords        = coords,
1010         ice_concentration = ice_concentration,
1011         min_gap       = min_gap,
1012         thickness      = thickness,
1013         duration_impact = duration_impact,
1014         time_first_impact = time_first_impact,
1015         time_step      = time_step,
1016         x_injection    = x_injection,
1017         Lpp            = Lpp,
1018         g              = g,
1019         output_dir     = OUTPUT_DIR,
1020     )
1021     all_files.append({
1022         "velocity_mps"      : v,
1023         "n_injected_rows"   : len(df_inj),
1024         "filepath"          : fp.name,
1025     })
1026
1027     summary_path = OUTPUT_DIR / "injection_files_summary.csv"
1028     pd.DataFrame(all_files).to_csv(summary_path, index=False)
1029
1030     print()
1031     print("Done. Summary:")
1032     print(pd.DataFrame(all_files).to_string(index=False))
1033     print(f"\nSummary CSV: {summary_path}")
1034
1035     elapsed = time.perf_counter() - t_start
1036     print(f"\nTotal run time : {int(elapsed // 60)} min {elapsed % 60:.1f} s")
1037
1038     plt.show()
```